MOBILE APPLICATION DEVELOPMENT

FINAL COMPLETE LAB EXAM BOOK

JAVA + XML • 59 QUESTIONS • THEORY • SIMPLE PROGRAM • MASTER PROGRAM • LAMBDA + NON-LAMBDA • VIVA • ERRORS

FINAL MERGED EDITION

This book consolidates the three previous study documents into one final exam textbook. It preserves the Java + XML, AndroidX/AppCompatActivity, findViewById(), reusable-pattern and non-SQLite approach.

Recommended Min SDK: 23 (Android 6.0). If your existing Android Studio project already uses another Min SDK and builds correctly, do not change it solely for the exam. Compile/target SDK should match the installed SDK used by your project.

Important: the Master Program is one reusable Android project containing multiple Java/XML files. Android does not realistically put Activities, Fragments, menus and layouts into one physical file.

Study order: Topic explanation → general syntax → lambda/non-lambda → simple program → final master program → 59 questions → viva → errors → brute-force recovery.

CONTENTS

PART 1 Theory and general syntax

PART 2 Lambda and non-lambda syntax

PART 3 Simple Master Program

PART 4 Final Master Program + all files

PART 5 Master variations for the 59 questions

PART 6 All 59 questions

PART 7 Viva questions and answers

PART 8 Common errors and fixes

PART 9 Brute-force recovery + final checklist

PART 1 — COMPLETE THEORY + GENERAL SYNTAX

The following section is the complete consolidated theory from the first study artifact, including Activity Lifecycle, widgets, validation, layouts, adapters, menus, fragments, IME, date/time, SharedPreferences, intents, master architecture, viva and errors.

MOBILE APPLICATION DEVELOPMENT

COMPLETE THEORY • JAVA + XML • ACTIVITY LIFECYCLE • GENERAL SYNTAX • VIVA • ERRORS • FIX

METHODS

Exam Textbook for MCA Mobile Application Development Lab

Source basis: your uploaded Android Java/XML notes, Fragment/Dynamic UI/Menus notes, lab-record material and model-question material. The source notes use Java + XML, AndroidX/AppCompatActivity and findViewById(). SQLite/database operations are excluded from this study guide.

How to use this book: Read the theory → memorize the general syntax → study Activity Lifecycle carefully → practice the reusable patterns → revise viva → use the error/fix chapter if you get stuck in the lab.

MASTER IDEA

You do not need to memorize every model question as a different program. Most questions reuse the same building blocks: XML UI → findViewById() → read values → validate → listener → store/display data → Intent/dialog/menu/list/fragment depending on the requirement.

CONTENTS

1. Android + Java/XML fundamentals

2. Project structure and Activity basics

3. Activity Lifecycle — IMPORTANT / COMPLETE

4. XML views and general widget syntax

5. Text fields, input types and validation

6. Layouts and general layout rules

7. Button clicks, Toast and AlertDialog

8. RadioButton, RadioGroup, CheckBox, Switch, RatingBar, SeekBar, ProgressBar

9. ListView, ArrayAdapter and custom adapters

10. Spinner and AutoCompleteTextView

11. ScrollView and Dynamic UI

12. Menus — Options, Context and Popup

13. Custom Dialog and PopupWindow

14. Fragments — concept, lifecycle, FragmentManager, transactions and data

15. IME / keyboard, DatePickerDialog and TimePickerDialog

16. SharedPreferences

17. Explicit and Implicit Intents — all important actions

18. General exam master architecture

19. Viva questions with detailed answers

20. Common errors and exact fixes

21. Brute-force exam recovery method

22. Final one-page revision

1. ANDROID + JAVA/XML FUNDAMENTALS

Android application development in this lab is based on a Java Activity class connected to an XML layout. XML describes the user interface; Java contains the program logic and event handling.

An Activity represents a screen in the application. Examples include Login, Home, Settings and Registration screens. The Android system manages an Activity through its lifecycle callbacks.

A View is a UI element such as TextView, EditText or Button. A ViewGroup is a container that can hold other Views, such as LinearLayout, FrameLayout or GridLayout.

A resource is an external project item such as an XML layout, string, color, drawable or menu. Android generates resource IDs such as R.id.button1 and R.layout.activity_main.

The uploaded notes use AppCompatActivity + AndroidX + findViewById() to keep the examples simple and lab-oriented.

Core Java/XML relationship:

activity_main.xml

↓

defines Button/EditText/TextView/etc.

↓

MainActivity.java

↓

setContentView(R.layout.activity_main)

↓

findViewById(R.id.someView)

↓

set listener / read value / update UI

General Activity syntax:

```
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
// Connect this Activity with the XML layout
setContentView(R.layout.activity_main);
}
}
```

Why super.onCreate()?

It calls the parent Activity implementation and allows Android/AppCompat to perform its normal Activity initialization. In an exam, keep it in the lifecycle method.

2. PROJECT STRUCTURE

Path Purpose

app/src/main/java/... Java source code and Activity/Fragment classes.

res/layout/ XML layouts for screens and UI.

res/values/strings.xml Text strings and localization resources.

res/values/colors.xml Color resources.

res/drawable/ Images, shapes and drawable resources.

res/menu/ Options/Popup menu XML files.

AndroidManifest.xml Declares Activities, permissions, app metadata and launcher Activity.

app/build.gradle Dependencies and Android build configuration.

General XML skeleton:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical">
<!-- Views go here -->
</LinearLayout>
```

Important XML attributes:

Attribute Meaning

layout_width / layout_height Size of the View. Common values: match_parent, wrap_content.

padding Space inside the View.

layout_margin Space outside the View.

gravity Alignment of content inside the View.

layout_gravity Position of the View inside its parent.

textSize Text size; use sp for text.

visibility visible, invisible or gone.

hint Placeholder shown in EditText.

layout_weight Shares remaining LinearLayout space.

3. ACTIVITY LIFECYCLE — MOST IMPORTANT TOPIC

The Activity lifecycle is the sequence of states and callback methods through which an Activity is created, becomes visible, becomes interactive, loses focus, becomes hidden, returns and is eventually destroyed. The Android system calls these methods at specific points.

The seven callbacks required in your lab are: onCreate(), onStart(), onResume(), onPause(), onStop(), onRestart(), onDestroy().

THE MAIN SEQUENCE

New Activity:

onCreate()

↓

onStart()

↓

onResume()

↓

Activity running / user interacting

Another Activity partially covers it:

onPause()

Activity becomes completely hidden:

onStop()

Stopped Activity comes back:

onRestart()

↓

onStart()

↓

onResume()

Activity is finally destroyed:

onDestroy()

3.1 onCreate()

When? Called when the Activity is first created. It is the normal place to initialize the screen.

What do we normally do? setContentView(), findViewById(), initialize variables, create adapters, set listeners and read initial state.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
EditText etName = findViewById(R.id.etName);
Button btnSave = findViewById(R.id.btnSave);
btnSave.setOnClickListener(v -> {
// button logic
});
}
```

Exam sentence: onCreate() initializes the Activity, connects the XML layout and prepares the UI.

3.2 onStart()

Called when the Activity becomes visible to the user. The Activity is about to come to the foreground.

```
@Override
protected void onStart() {
super.onStart();
Log.d("LIFECYCLE", "onStart");
}
```

3.3 onResume()

Called when the Activity starts interacting with the user. The Activity is in the foreground and ready for user input.

```
@Override
protected void onResume() {
super.onResume();
Log.d("LIFECYCLE", "onResume");
}
```

3.4 onPause()

Called when another Activity or window is about to come in front. The source notes describe it as a place to save light/unsaved changes and release resources that consume CPU.

```
@Override
protected void onPause() {
super.onPause();
Log.d("LIFECYCLE", "onPause");
}
```

3.5 onStop()

Called when the Activity is no longer visible. The Activity is in the background.

```
@Override
protected void onStop() {
super.onStop();
Log.d("LIFECYCLE", "onStop");
}
```

3.6 onRestart()

Called when a stopped Activity is coming back. It is followed by onStart() and onResume().

```
@Override
protected void onRestart() {
super.onRestart();
Log.d("LIFECYCLE", "onRestart");
}
```

3.7 onDestroy()

Called before the Activity is destroyed. The source notes describe it as a place for cleanup of resources and held references. For important persistent data, do not depend on onDestroy() as your only saving mechanism.

```
@Override
protected void onDestroy() {
super.onDestroy();
Log.d("LIFECYCLE", "onDestroy");
}
```

3.8 Complete lifecycle program for the lab

```
package com.example.myapp;
import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
private static final String TAG = "LIFECYCLE";
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
Log.d(TAG, "onCreate");
}
@Override
protected void onStart() {
super.onStart();
Log.d(TAG, "onStart");
}
@Override
protected void onResume() {
super.onResume();
Log.d(TAG, "onResume");
}
@Override
protected void onPause() {
super.onPause();
Log.d(TAG, "onPause");
}
@Override
protected void onStop() {
super.onStop();
Log.d(TAG, "onStop");
}
@Override
protected void onRestart() {
super.onRestart();
Log.d(TAG, "onRestart");
}
@Override
protected void onDestroy() {
super.onDestroy();
```

Log.d(TAG, "onDestroy");
}
}

3.9 Lifecycle sequences to memorize

Situation Typical callback flow
Open Activity for first time onCreate → onStart → onResume
Activity is partially covered onPause
Activity becomes fully hidden onStop
Stopped Activity returns onRestart → onStart → onResume
Activity finishes onPause → onStop → onDestroy (typical)

3.10 How to demonstrate it in the lab

Run the app and open Logcat. Filter by the TAG LIFECYCLE. Open the Activity and observe onCreate/onStart/onResume. Navigate away and observe onPause/onStop. Return to the stopped Activity and observe onRestart/onStart/onResume.

3.11 Important distinction

onPause() means the Activity is losing foreground focus. onStop() means it is no longer visible. onRestart() occurs when a previously stopped Activity is returning. onDestroy() means the Activity is being destroyed, but you should not treat it as a guaranteed location for critical persistent storage.

4. XML VIEWS AND GENERAL WIDGET SYNTAX

A View is identified in XML using android:id and retrieved in Java using findViewById().
TextView
<TextView
android:id="@+id/tvTitle"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello"
android:textSize="22sp" />
TextView tvTitle = findViewById(R.id.tvTitle);
tvTitle.setText("New title");
String value = tvTitle.getText().toString();
EditText
<EditText
android:id="@+id/etName"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter name"
android:inputType="textPersonName" />
String name = etName.getText().toString().trim();
etName.setText("Akhil");
Button and click
Button btn = findViewById(R.id.btnSubmit);
btn.setOnClickListener(v -> {
Toast.makeText(this, "Clicked", Toast.LENGTH_SHORT).show();
});
Common inputType values
inputType Use
text Normal text.
textPersonName Names.
textEmailAddress Email input.

textPassword Password input.

number / numberDecimal / numberSigned Numeric input.

phone Phone number.

textMultiLine Multiline text.

date / time Date/time entry.

5. TEXT FIELDS AND VALIDATION

Validation means checking user input before accepting or processing it.

The most common exam pattern is: read → check empty → setError/requestFocus → return → continue if valid.

String name = etName.getText().toString().trim();

if (name.isEmpty()) {

etName.setError("Name is required");

etName.requestFocus();

return;

}

Email validation:

```
private boolean isValidEmail(String email) {
```

return android.util.Patterns.EMAIL_ADDRESS

.matcher(email)

.matches();

}

Indian 10-digit mobile pattern from the notes:

```
private boolean isValidPhone(String phone) {
```

return phone.matches("[6-9][0-9]{9}");

}

Strong password pattern from the notes:

```
private boolean isStrongPassword(String pwd) {
```

return pwd.matches(

"^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=!]).{8,}$"

);

}

Typical validation chain:

if (name.isEmpty()) {

etName.setError("Name required");

etName.requestFocus();

} else if (!isValidEmail(email)) {

etEmail.setError("Enter valid email");

etEmail.requestFocus();

} else if (!isValidPhone(phone)) {

etPhone.setError("Enter valid mobile");

etPhone.requestFocus();

} else {

Toast.makeText(this,

"Valid input",

Toast.LENGTH_SHORT).show();

}

6. LAYOUTS — THEORY + GENERAL SYNTAX

LinearLayout: arranges children vertically or horizontally.

```
<LinearLayout
```

android:layout_width="match_parent"

android:layout_height="match_parent"

android:orientation="vertical">

```
<!-- children -->
```

```
</LinearLayout>
```

RelativeLayout: positions views relative to the parent or other views. It appears in the original notes as a common layout pattern.

ConstraintLayout: positions views using constraints. Useful for responsive layouts and appears in the practice/model questions.

FrameLayout: stacks children; commonly used as a container for fragments.

```
<FrameLayout
android:id="@+id/fragmentContainer"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

TableLayout: organizes content into rows using TableRow. Useful for structured forms.

```
<TableLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:stretchColumns="1">
<TableRow>
<TextView android:text="Name" />
<EditText android:hint="Enter name" />
</TableRow>
</TableLayout>
```

GridLayout: arranges children in rows and columns.

```
<GridLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:columnCount="2">
<!-- buttons/views -->
</GridLayout>
```

ScrollView: allows vertical scrolling. It normally contains one direct child, commonly a LinearLayout.

```
<ScrollView
android:layout_width="match_parent"
android:layout_height="match_parent">
<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical">
<!-- many views -->
</LinearLayout>
</ScrollView>
```

Common layout mistake: putting multiple direct children inside a ScrollView. Use one parent container and place the other Views inside it.

7. TOAST, ALERTDIALOG AND BUTTON ACTIONS

Toast: small temporary message that disappears automatically and does not take focus.

```
Toast.makeText(
this,
"Saved!",
Toast.LENGTH_SHORT
).show();
```

AlertDialog: modal dialog used for confirmation, messages or choices.

```
new AlertDialog.Builder(this)
.setTitle("Confirm")
.setMessage("Do you want to continue?")
.setPositiveButton("Yes", (dialog, which) -> {
// Yes action
})
.setNegativeButton("No", null)
```

.show();
Single-choice AlertDialog:
String[] modes = {"Light", "Dark", "Blue"};
new AlertDialog.Builder(this)
.setTitle("Choose Theme")
.setSingleChoiceItems(modes, 0,
(dialog, which) -> {
Toast.makeText(this,
modes[which],
Toast.LENGTH_SHORT).show();
dialog.dismiss();
})
.show();
RadioButton vs CheckBox
RadioButton CheckBox
Usually one choice within a RadioGroup. Multiple independent choices can be selected.
Read using getCheckedRadioButtonId(). Read using isChecked().

8. IMPORTANT FORM WIDGETS

RadioGroup + RadioButton:
int id = rgGender.getCheckedRadioButtonId();
if (id == -1) {
Toast.makeText(this,
"Select gender",
Toast.LENGTH_SHORT).show();
} else {
RadioButton rb = findViewById(id);
String gender = rb.getText().toString();
}
CheckBox:
if (cbJava.isChecked()) {
// Java selected
}
if (cbPython.isChecked()) {
// Python selected
}
Switch:
swNotifications.setOnCheckedChangeListener(
(buttonView, isChecked) -> {
Toast.makeText(this,
isChecked ? "Enabled" : "Disabled",
Toast.LENGTH_SHORT).show();
}
);
RatingBar:
float rating = ratingBar.getRating();
ratingBar.setOnRatingBarChangeListener(
(bar, value, fromUser) -> {
Toast.makeText(this,
"Rating: " + value,
Toast.LENGTH_SHORT).show();
}
);
SeekBar:

```
seekBar.setOnSeekBarChangeListener(
new SeekBar.OnSeekBarChangeListener() {
 @Override
public void onProgressChanged(
SeekBar bar,
int progress,
boolean fromUser) {
// use progress
}
 @Override public void onStartTrackingTouch(SeekBar bar) {}
 @Override public void onStopTrackingTouch(SeekBar bar) {}
}
);
ProgressBar:
progressBar.setProgress(60);
progressBar.setVisibility(View.GONE);
```

9. LISTVIEW + ARRAYADAPTER

A ListView displays a list of items. An Adapter acts as the bridge between the data and the ListView.

The simplest exam pattern uses an ArrayList and ArrayAdapter.

```
ArrayList items = new ArrayList<>();
items.add("Java");
items.add("Android");
items.add("XML");
ArrayAdapter adapter =
new ArrayAdapter<>(
this,
android.R.layout.simple_list_item_1,
items
);
ListView listView = findViewById(R.id.listView);
listView.setAdapter(adapter);
Item click:
listView.setOnItemClickListener(
(parent, view, position, id) -> {
String selected =
items.get(position);
Toast.makeText(
this,
selected,
Toast.LENGTH_SHORT
).show();
}
);
Adding and deleting items:
items.add("New item");
adapter.notifyDataSetChanged();
items.remove(position);
adapter.notifyDataSetChanged();
```

Why notifyDataSetChanged()?

It tells the Adapter that the underlying data has changed and the ListView should refresh its displayed contents.

Custom Adapter concept:

For custom rows, create a row XML and a custom Adapter extending BaseAdapter or another suitable adapter class. In getView(), inflate/reuse the row, find its views, put the data into them and return the row View.

```
@Override
public View getView(
int position,
View convertView,
ViewGroup parent) {
if (convertView == null) {
convertView = LayoutInflater.from(context)
.inflate(R.layout.row_student, parent, false);
}
TextView tvName =
convertView.findViewById(R.id.tvName);
Student s = list.get(position);
tvName.setText(s.getName());
return convertView;
}
```

10. SPINNER + AUTOCOMPLETETEXTVIEW

Spinner is a drop-down selection control.

```
String[] courses = {
"MCA", "MBA", "BCA", "MSc"
};
Spinner spinner = findViewById(R.id.spinner);
ArrayAdapter adapter =
new ArrayAdapter<>(
this,
android.R.layout.simple_spinner_item,
courses
);
adapter.setDropDownViewResource(
android.R.layout.simple_spinner_dropdown_item
);
spinner.setAdapter(adapter);
spinner.setOnItemSelectedListener(
new AdapterView.OnItemSelectedListener() {
@Override
public void onItemSelected(
AdapterView parent,
View view,
int position,
long id) {
String value = courses[position];
}
@Override
public void onNothingSelected(
AdapterView parent) {
}
}
);
```

AutoCompleteTextView:

```
String[] cities = {
"Kochi", "Kollam", "Kannur"
};
ArrayAdapter adapter =
new ArrayAdapter<>(
```

this,
android.R.layout.simple_dropdown_item_1line,
cities
);
AutoCompleteTextView actv =
findViewById(R.id.cityAutoCompleteTextView);
actv.setAdapter(adapter);
actv.setThreshold(1);

11. SCROLLVIEW + DYNAMIC UI + TEXTWATCHER

Dynamic UI means creating/modifying/removing Views in Java at runtime instead of defining every View in XML.
LinearLayout container =
findViewById(R.id.container);
TextView tv = new TextView(this);
tv.setText("Created in Java");
tv.setTextSize(20);
tv.setPadding(16, 16, 16, 16);
container.addView(tv);
Dynamic EditText:
EditText field = new EditText(this);
field.setHint("Enter value");
LinearLayout.LayoutParams params =
new LinearLayout.LayoutParams(
ViewGroup.LayoutParams.MATCH_PARENT,
ViewGroup.LayoutParams.WRAP_CONTENT
);
field.setLayoutParams(params);
container.addView(field);
Remove Views:
container.removeView(view);
container.removeAllViews();
int count = container.getChildCount();
View child = container.getChildAt(0);
TextWatcher for live character count:
editText.addTextChangedListener(
new TextWatcher() {
@Override
public void beforeTextChanged(
CharSequence s,
int start,
int count,
int after) {
}
@Override
public void onTextChanged(
CharSequence s,
int start,
int before,
int count) {
textView.setText(
"Characters: " + s.length()
);
}
@Override

```
public void afterTextChanged(
Editable s) {
}
}
);
```

12. MENUS — OPTIONS, CONTEXT AND POPUP

Android traditional View applications use three important menu patterns in your notes: Options Menu, Context Menu and Popup Menu.

Options Menu = Activity-level actions/settings. Context Menu = actions for a selected View, usually after long press. Popup Menu = small contextual menu anchored to a View.

Options Menu XML:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
<item
android:id="@+id/action_settings"
android:title="Settings" />
<item
android:id="@+id/action_save"
android:title="Save" />
</menu>
```

Options Menu Java:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
getMenuInflater().inflate(
R.menu.main_menu,
menu
);
return true;
}
@Override
public boolean onOptionsItemSelected(MenuItem item) {
if (item.getItemId() == R.id.action_save) {
Toast.makeText(this,
"Saved",
Toast.LENGTH_SHORT).show();
return true;
}
return super.onOptionsItemSelected(item);
}
```

Context Menu:

```
registerForContextMenu(listView);
@Override
public void onCreateContextMenu(
ContextMenu menu,
View v,
ContextMenu.ContextMenuInfo info) {
super.onCreateContextMenu(menu, v, info);
menu.setHeaderTitle("Choose Action");
menu.add(0, 1, 0, "Edit");
menu.add(0, 2, 1, "Delete");
}
@Override
public boolean onContextItemSelected(MenuItem item) {
if (item.getItemId() == 2) {
// delete
```

return true;
}
return super.onContextItemSelected(item);
}
Popup Menu:
Button more = findViewById(R.id.btnMore);
more.setOnClickListener(v -> {
PopupMenu popup =
new PopupMenu(MainActivity.this, v);
popup.getMenuInflater().inflate(
R.menu.popup_menu,
popup.getMenu()
);
popup.setOnMenuItemClickListener(item -> {
Toast.makeText(
this,
item.getTitle(),
Toast.LENGTH_SHORT
).show();
return true;
});
popup.show();
});
Menu comparison:
Menu Purpose Trigger
Options Menu Activity-level actions/settings. App bar / overflow.
Context Menu Actions for a selected View. Usually long press.
Popup Menu Small contextual actions. Tap an anchor View.
Dynamic menu state:
```
@Override
```
public boolean onPrepareOptionsMenu(Menu menu) {
MenuItem save = menu.findItem(R.id.action_save);
save.setVisible(isFormValid);
return super.onPrepareOptionsMenu(menu);
}
// Call when menu state changes:
invalidateOptionsMenu();

13. CUSTOM DIALOG + POPUPWINDOW

A custom dialog uses a custom XML layout inside an AlertDialog. It is useful for forms such as Add Item.
View view = getLayoutInflater().inflate(
R.layout.dialog_add_item,
null
);
EditText etName =
view.findViewById(R.id.etName);
new AlertDialog.Builder(this)
.setTitle("Add Item")
.setView(view)
.setPositiveButton("Save",
(dialog, which) -> {
String name =
etName.getText().toString();
})

```
.setNegativeButton("Cancel", null)
.show();
```

PopupWindow is a custom floating View. The notes show showAsDropDown(anchor) and showAtLocation(...).

```
View popupView =
getLayoutInflater().inflate(
R.layout.popup_layout, null);
PopupWindow popupWindow =
new PopupWindow(
popupView,
ViewGroup.LayoutParams.WRAP_CONTENT,
ViewGroup.LayoutParams.WRAP_CONTENT,
true
);
popupWindow.showAsDropDown(
button, 0, 10
);
```

14. FRAGMENTS — COMPLETE THEORY

A Fragment is a reusable portion of UI with its own layout and lifecycle, hosted inside an Activity. Fragments are useful for flexible screens, navigation and reusable sections.

Your uploaded notes use AndroidX Fragment classes for their main Fragment examples.

Fragment lifecycle from the notes:

```
onAttach()
↓
onCreate()
↓
onCreateView()
↓
onViewCreated()
↓
onStart()
↓
onResume()
↓
onPause()
↓
onStop()
↓
onDestroyView()
↓
onDestroy()
↓
onDetach()
```

Meaning:

Callback Purpose

onAttach() Fragment is attached to the Activity/context.

onCreate() Initialize non-UI state.

onCreateView() Inflate/create and return the Fragment layout.

onViewCreated() View hierarchy is ready; find views and attach listeners.

onStart()/onResume() Visible / active and interactive.

onPause()/onStop() Leaving foreground / no longer visible.

onDestroyView() Fragment's View hierarchy is destroyed.

onDestroy() Fragment object is being destroyed.

onDetach() Fragment is detached from Activity.

Basic Fragment:

```
public class HomeFragment extends Fragment {
public HomeFragment() {
// Empty constructor
}
@Override
public View onCreateView(
LayoutInflater inflater,
ViewGroup container,
Bundle savedInstanceState) {
return inflater.inflate(
R.layout.fragment_home,
container,
false
);
}
}
```

Dynamic Fragment container:

```
<androidx.fragment.app.FragmentContainerView
android:id="@+id/container"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

Replace Fragment:

```
getSupportFragmentManager()
.beginTransaction()
.replace(
R.id.container,
new HomeFragment()
)
.commit();
```

Back stack:

```
getSupportFragmentManager()
.beginTransaction()
.replace(
R.id.container,
new ProfileFragment()
)
.addToBackStack(null)
.commit();
```

Why addToBackStack()?

It records the Fragment transaction so the user can use Back to return to the previous Fragment state.

Fragment data using Bundle:

```
Bundle bundle = new Bundle();
bundle.putString("name", "Akhil");
DetailsFragment fragment =
new DetailsFragment();
fragment.setArguments(bundle);
getSupportFragmentManager()
.beginTransaction()
.replace(R.id.container, fragment)
.commit();
```

Receive it:

```
Bundle bundle = getArguments();
if (bundle != null) {
```

String name =
bundle.getString("name");
}
Important exam distinction: Activity-to-Activity data is commonly passed through Intent extras/Bundle.
Activity-to-Fragment data is commonly passed through Fragment arguments using
setArguments()/getArguments().

15. IME / KEYBOARD + DATE + TIME

IME means Input Method Editor, i.e. the on-screen keyboard.
IME options change the action button shown on the keyboard, such as Search, Done, Next, Go, Send or Previous.

```
<EditText
android:id="@+id/etSearch"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Search..."
android:inputType="text"
android:imeOptions="actionSearch"
android:maxLines="1" />
etSearch.setOnEditorActionListener(
(v, actionId, event) -> {
if (actionId ==
EditorInfo.IME_ACTION_SEARCH) {
String q =
etSearch.getText().toString();
Toast.makeText(
this,
"Searching: " + q,
Toast.LENGTH_SHORT
).show();
return true;
}
return false;
}
);
Hide keyboard:
InputMethodManager imm =
(InputMethodManager)
getSystemService(Context.INPUT_METHOD_SERVICE);
imm.hideSoftInputFromWindow(
etSearch.getWindowToken(),
0
);
DatePickerDialog:
Calendar calendar = Calendar.getInstance();
DatePickerDialog dialog =
new DatePickerDialog(
this,
(view, year, month, day) -> {
String date =
day + "/" + (month + 1) + "/" + year;
tvDate.setText(date);
},
calendar.get(Calendar.YEAR),
calendar.get(Calendar.MONTH),
```

calendar.get(Calendar.DAY_OF_MONTH)
);
dialog.show();
Prevent past dates when required:
dialog.getDatePicker().setMinDate(
System.currentTimeMillis()
);
TimePickerDialog:
Calendar calendar = Calendar.getInstance();
TimePickerDialog dialog =
new TimePickerDialog(
this,
(view, hour, minute) -> {
String time =
String.format(
Locale.getDefault(),
"%02d:%02d",
hour,
minute
);
tvTime.setText(time);
},
calendar.get(Calendar.HOUR_OF_DAY),
calendar.get(Calendar.MINUTE),
true
);
dialog.show();

16. SHAREDPREFERENCES

SharedPreferences stores small amounts of persistent application data as key-value pairs. Typical lab uses include saving a name, email, remember-me flag or login session state.
Create/open preferences:
SharedPreferences sp =
getSharedPreferences(
"UserData",

MODE_PRIVATE

);
Write data:
sp.edit()
.putString("name", "Akhil")
.putString("email", "a@example.com")
.putBoolean("remember", true)
.apply();
Read data:
String name =
sp.getString("name", "Not saved");
boolean remember =
sp.getBoolean("remember", false);
Clear all data:
sp.edit().clear().apply();
Remember-me login pattern:
SharedPreferences sp =
getSharedPreferences(

"Session",

MODE_PRIVATE

);
if (sp.getBoolean("isLoggedIn", false)) {
startActivity(
new Intent(this, HomeActivity.class)
);
finish();
return;
}
// after successful login:
sp.edit()
.putBoolean("isLoggedIn", true)
.putString("user", username)
.apply();
Security note from the source: do not store real passwords in plain SharedPreferences. For sensitive data, the notes recommend appropriate secure storage such as encrypted preferences/Android Keystore. For your exam, SharedPreferences is primarily a small key-value persistence concept.

17. INTENTS — EXPLICIT + IMPLICIT

An Intent is a messaging object used to request an action from another Android application component. The notes distinguish Explicit and Implicit Intents.
Explicit Intent:
The exact target component class is named. Typical use: Activity → Activity.
Intent intent = new Intent(
MainActivity.this,
WelcomeActivity.class
);
intent.putExtra("user", user);
startActivity(intent);
Receive:
String user =
getIntent().getStringExtra("user");
Bundle version:
Bundle bundle = new Bundle();
bundle.putString("user", user);
Intent intent =
new Intent(
MainActivity.this,
WelcomeActivity.class
);
intent.putExtras(bundle);
startActivity(intent);
Implicit Intent:
The action/data is described, and Android finds a suitable application/component that can handle it.
Browser / Website:
Intent intent = new Intent(
Intent.ACTION_VIEW,
Uri.parse("https://www.google.com")
);
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);

```
}
Dialer:
Intent intent = new Intent(
Intent.ACTION_DIAL,
Uri.parse("tel:9876543210")
);
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);
}
```

SMS:

```
Intent intent = new Intent(
Intent.ACTION_SENDTO,
Uri.parse("sms:9876543210")
);
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);
}
Email:
Intent intent =
new Intent(Intent.ACTION_SENDTO);
intent.setData(
Uri.parse("mailto:example@gmail.com")
);
intent.putExtra(
Intent.EXTRA_SUBJECT,
```

"MAD"

```
);
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);
}
Maps:
Uri location =
Uri.parse("geo:0,0?q=Kochi");
Intent intent =
new Intent(
Intent.ACTION_VIEW,
location
);
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);
}
Calendar:
Intent intent =
new Intent(Intent.ACTION_INSERT);
intent.setData(
Uri.parse(
"content://com.android.calendar/events"
)
);
```

```
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);
}
```

Share text:

```
Intent intent =
new Intent(Intent.ACTION_SEND);
intent.setType("text/plain");
intent.putExtra(
Intent.EXTRA_TEXT,
"Hello from MAD"
);
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(
Intent.createChooser(
intent,
"Share using"
)
);
}
```

Why resolveActivity()?

It checks whether an installed application can handle the Intent before calling startActivity(). This avoids an ActivityNotFoundException when no suitable application exists.

18. EXAM MASTER ARCHITECTURE

The uploaded question bank explicitly identifies a reusable architecture behind the integrated model questions.

ACTIVITY 1

↓

Form widgets

↓

Validation

↓

Options Menu / Button

↓

AlertDialog

↓

SharedPreferences

↓

Bundle

↓

Explicit Intent

↓

ACTIVITY 2

↓

Read Intent data / preferences

↓

ListView + Adapter

↓

Normal click

→ AlertDialog / Toast

↓

Long press

→ Context Menu

↓

Implicit Intent

→ Browser / Dialer / Email / Maps / Calendar

Reusable replacement rule:

Keep the pattern Change for the question

EditText + validation Name, email, patient, destination, product, username, etc.

RadioGroup Gender, intensity, room type, category, etc.

CheckBox Hobbies, facilities, warnings, features, add-ons.

Spinner Course, state, country, vehicle, room, category.

ListView + adapter Products, tasks, contacts, courses, bookings, books.

Intent destination Browser, dialer, SMS, email, maps, calendar.

Messages Question-specific validation, confirmation and Toast text.

This is why learning patterns is more efficient than memorizing dozens of unrelated programs.

19. VIVA QUESTIONS + DETAILED ANSWERS

These answers are designed to be spoken in a viva: first give the direct answer, then add the explanation if the examiner asks further.

Q1. What is an Activity?

Answer: An Activity represents a screen in an Android application. It has a lifecycle managed by the Android system and can contain Views such as TextView, EditText and Button.

Q2. What is the Activity lifecycle?

Answer: It is the sequence of callback methods through which an Activity is created, becomes visible, becomes interactive, loses focus, becomes hidden, returns and is destroyed. The important callbacks are onCreate, onStart, onResume, onPause, onStop, onRestart and onDestroy.

Q3. Which method is normally used to set the XML layout?

Answer: setContentView(R.layout.activity_main) is normally called in onCreate().

Q4. Difference between onPause() and onStop()?

Answer: onPause() occurs when the Activity is losing foreground focus. onStop() occurs when the Activity is no longer visible.

Q5. When is onRestart() called?

Answer: When a previously stopped Activity is coming back. It is followed by onStart() and onResume().

Q6. What is Log.d() used for?

Answer: It writes debug messages to Logcat. It is useful for observing lifecycle callbacks and debugging program flow.

Q7. What is findViewById()?

Answer: It finds a View from the currently loaded XML layout using its resource ID and returns a reference that Java can use.

Q8. What is the difference between match_parent and wrap_content?

Answer: match_parent asks the View to use the available parent dimension; wrap_content asks it to use only the space required by its content.

Q9. Why use dp and sp?

Answer: dp is commonly used for dimensions and spacing; sp is used for text size so text can respect user font scaling.

Q10. What is validation?

Answer: Validation checks whether user input satisfies the required conditions before processing it.

Q11. What does setError() do?

Answer: It displays an error indicator/message associated with the input field. requestFocus() can move focus to that field.

Q12. RadioButton vs CheckBox?

Answer: RadioButtons inside a RadioGroup normally provide one choice; CheckBoxes allow independent multiple selections.

Q13. What does getCheckedRadioButtonId() return?

Answer: It returns the ID of the selected RadioButton. If none is selected, it returns -1.

Q14. What is an Adapter?

Answer: An Adapter connects data to a UI component such as ListView or Spinner and supplies the Views/items that should be displayed.

Q15. Why notifyDataSetChanged()?

Answer: After changing the underlying list, notifyDataSetChanged() tells the Adapter that the data changed so the displayed list can refresh.

Q16. What is a Spinner?

Answer: A Spinner is a drop-down selection widget. It is commonly populated using an ArrayAdapter.

Q17. What is ScrollView?

Answer: It provides scrolling for content larger than the available screen area. A common pattern is one LinearLayout child containing many Views.

Q18. What is dynamic UI?

Answer: Creating, modifying or removing Views programmatically at runtime rather than defining every View in

XML.

Q19. What is an Intent?

Answer: An Intent is a messaging object used to request an action from another Android component.

Q20. Explicit vs Implicit Intent?

Answer: Explicit Intent names the exact target component, such as WelcomeActivity. Implicit Intent describes an action/data and lets Android find a suitable handler, such as a browser for a website.

Q21. Why use resolveActivity()?

Answer: To check whether there is an installed component capable of handling the Intent before starting it.

Q22. What is Bundle?

Answer: Bundle is a key-value container commonly used to carry small pieces of data such as String/int values between Android components.

Q23. What is SharedPreferences?

Answer: A simple persistent key-value storage mechanism for small application data.

Q24. Why use apply()?

Answer: It commits SharedPreferences changes asynchronously. It is commonly used in the lab examples after editing preferences.

Q25. What is a Fragment?

Answer: A Fragment is a reusable portion of UI with its own lifecycle, hosted inside an Activity.

Q26. replace() vs addToBackStack()?

Answer: replace() changes the Fragment displayed in a container. addToBackStack() records the transaction so Back can return to the previous Fragment state.

Q27. setArguments() / getArguments()?

Answer: setArguments() attaches a Bundle to a Fragment before it is displayed. getArguments() retrieves that Bundle inside the Fragment.

Q28. Options Menu vs Context Menu vs Popup Menu?

Answer: Options Menu is for Activity-level actions, Context Menu is usually triggered by long press on a View, and Popup Menu is a small menu anchored to a View.

Q29. What is TextWatcher?

Answer: It observes text changes in an editable text field. It provides beforeTextChanged(), onTextChanged() and afterTextChanged().

Q30. What is IME?

Answer: Input Method Editor, referring to the on-screen keyboard/input method. imeOptions can control actions such as Search or Done.

Q31. DatePickerDialog vs TimePickerDialog?

Answer: DatePickerDialog lets the user choose a date; TimePickerDialog lets the user choose a time.

Q32. Why use finish() after redirecting from Login?

Answer: It removes the current Activity from the back stack so the user does not return to the login screen using Back.

Q33. What is an AlertDialog used for?

Answer: Confirmation, messages, single-choice/multi-choice selection and simple user decisions.

Q34. What is a Toast?

Answer: A short temporary message that disappears automatically.

Q35. What is the purpose of AndroidManifest.xml?

Answer: It declares application components and metadata such as Activities and required permissions, and identifies the launcher Activity.

20. COMMON ERRORS + EXACT FIX METHODS

This section is designed for the practical exam. If the program does not work, identify the category first instead of rewriting everything.

ERROR 1 — Cannot find symbol R.id.xyz

Cause: XML ID is different, missing, or the resource XML has another error.

Fix: Check the exact spelling in XML and Java. Make sure the View has android:id="@+id/xyz". Fix any red XML error first, then rebuild.

XML:

android:id="@+id/btnSubmit"

Java:

findViewById(R.id.btnSubmit);

ERROR 2 — NullPointerException after findViewById()

Cause: Wrong ID, wrong layout loaded, or findViewById() is called before setContentView().

Fix: setContentView() first → then findViewById(). Verify the ID exists in that exact layout.

setContentView(R.layout.activity_main);

Button btn =

findViewById(R.id.btnSubmit);

ERROR 3 — Activity cannot be opened

Cause: The second Activity may not be declared correctly in the manifest or the class name/package may be wrong.

Fix: Check AndroidManifest.xml and the Activity class name/package.

```
<activity
```

android:name=".WelcomeActivity"

android:exported="false" />

ERROR 4 — App crashes when opening website/dialer/email

Cause: No installed application can handle the Intent.

Fix: Use resolveActivity() before startActivity().

if (intent.resolveActivity(

getPackageManager()) != null) {

startActivity(intent);

} else {

Toast.makeText(

this,

"No suitable app found",

Toast.LENGTH_SHORT

).show();

}

ERROR 5 — RadioButton value not received

Cause: No RadioButton is selected, or the selected ID is not checked.

Fix: Test for -1 first.

int id =

radioGroup.getCheckedRadioButtonId();

if (id == -1) {

Toast.makeText(

this,

"Select one option",

Toast.LENGTH_SHORT

).show();
return;
}
RadioButton rb = findViewById(id);
ERROR 6 — ListView does not update after delete/add
Cause: The ArrayList changed but the Adapter was not notified.
Fix: Call adapter.notifyDataSetChanged().
items.remove(position);
adapter.notifyDataSetChanged();
ERROR 7 — ListView is empty
Cause: Items were not added, adapter was not created correctly, or setAdapter() was forgotten.
Fix: Verify this order: create list → add items → create adapter → setAdapter().
ERROR 8 — Spinner not showing values
Cause: Adapter not attached.
Fix: spinner.setAdapter(adapter). Also set the drop-down resource if required.
ERROR 9 — Spinner callback not working
Cause: OnItemSelectedListener not attached or wrong callback implementation.
Fix: Implement both onItemSelected() and onNothingSelected().
ERROR 10 — ScrollView content not scrolling
Cause: Wrong height/structure or multiple direct children.
Fix: Use one direct child such as a vertical LinearLayout with wrap_content height.
ERROR 11 — Dynamic View does not appear
Cause: View was created but never added to a visible parent.
Fix: configure View → optional LayoutParams → parent.addView(view).
ERROR 12 — Fragment crashes because view is null
Cause: findViewById() was performed on the wrong object or before the Fragment view was created.
Fix: Inflate the Fragment layout in onCreateView() and use the returned view / use onViewCreated().
```
@Override
public View onCreateView(
LayoutInflater inflater,
ViewGroup container,
Bundle savedInstanceState) {
View view = inflater.inflate(
R.layout.fragment_home,
container,
false
);
TextView tv =
view.findViewById(R.id.tvTitle);
return view;
}
```
ERROR 13 — Fragment transaction does nothing
Cause: Wrong container ID, transaction not committed, or mixed Fragment APIs.
Fix: Use a valid FragmentContainerView/FrameLayout ID, call commit(), and do not mix android.app.Fragment with androidx.fragment.app.Fragment.
ERROR 14 — SharedPreferences value not saved
Cause: Wrong key, wrong preference file, or edit() was not applied.
Fix: Check getSharedPreferences name and keys. End writes with apply().
ERROR 15 — getString() returns default value
Cause: The key was never saved or the key name does not exactly match.
Fix: Compare the putString() key and getString() key character-by-character.
ERROR 16 — TextWatcher causes unexpected behavior
Cause: Validation/update code may run on every keystroke.
Fix: Keep live-update logic small; choose the correct callback. Use afterTextChanged() when you need the final

Editable value.
ERROR 17 — DatePicker allows past dates
Fix: Apply setMinDate(System.currentTimeMillis()) when the question requires future dates only.
ERROR 18 — IME Done/Search not triggering
Cause: imeOptions and action ID do not match.
Fix: XML actionSearch/actionDone must match EditorInfo.IME_ACTION_SEARCH/IME_ACTION_DONE.
ERROR 19 — App does not compile after adding a menu
Cause: Menu XML ID does not exist or Java references a different ID.
Fix: Compare res/menu XML IDs with R.id references. Also ensure the file is inside res/menu/.
ERROR 20 — Wrong FragmentManager import
Cause: Mixing platform and AndroidX Fragment APIs.
Fix: If using AndroidX Fragment, use androidx.fragment.app.Fragment, FragmentManager and FragmentTransaction consistently.

21. BRUTE-FORCE EXAM RECOVERY METHOD

If you are in the practical exam and your program is not working, do not panic. Use this deterministic recovery sequence.
STEP 1 — Make the app compile first
Remove optional features temporarily. Keep only Activity + XML + one Button. Build/run. Once it works, add the required feature.
STEP 2 — Check the three basic lines

```
setContentView(R.layout.activity_main);
Button btn =
findViewById(R.id.btnSubmit);
btn.setOnClickListener(v -> {
Toast.makeText(
this,
"Working",
Toast.LENGTH_SHORT
).show();
});
```

If this works, your Activity/XML connection is correct.
STEP 3 — Add one concept at a time
Form
→ validation
→ adapter
→ dialog
→ SharedPreferences
→ Intent
→ second Activity
→ menu
→ long click
→ implicit Intent
→ fragment
STEP 4 — If IDs fail
Copy the ID directly from XML and use R.id.. Never type a similar name from memory.
STEP 5 — If navigation fails
Use the smallest Explicit Intent first:

```
startActivity(
new Intent(
MainActivity.this,
SecondActivity.class
)
);
```

Then add extras only after basic navigation works.

STEP 6 — If external Intent fails

Use resolveActivity(). If there is no handler, show a Toast instead of allowing a crash.

STEP 7 — If ListView fails

Use the simplest ArrayAdapter first. Do not start with a custom Adapter unless the question requires custom rows.

```
ArrayList list =
new ArrayList<>();
list.add("One");
list.add("Two");
ArrayAdapter adapter =
new ArrayAdapter<>(
this,
android.R.layout.simple_list_item_1,
list
);
listView.setAdapter(adapter);
```

STEP 8 — If Fragment fails

Use a single FragmentContainerView and a single replace(). Check imports carefully.

STEP 9 — If the examiner asks for more features

Do not rewrite the entire program. Add the feature using the known pattern. For example, Context Menu = registerForContextMenu → onCreateContextMenu → onContextItemSelected.

STEP 10 — Final 30-second checklist

[ ] XML file name correct
[ ] setContentView() correct
[ ] Every Java ID exists in XML
[ ] Listener attached
[ ] Validation returns on invalid input
[ ] Adapter attached
[ ] notifyDataSetChanged() after list change
[ ] Activity declared in Manifest
[ ] Intent target class correct
[ ] resolveActivity() for external Intent
[ ] Fragment imports are consistent
[ ] SharedPreferences keys match
[ ] Build again before demonstration

BRUTE-FORCE RULE: Compile → Run → Verify one feature → Add next feature → Compile again. Do not add ten features and debug ten problems at once.

22. FINAL ONE-PAGE REVISION

MEMORIZE THESE PATTERNS

1. Activity:

```
setContentView(R.layout.activity_main);
```

2. View:

```
Button btn = findViewById(R.id.btn);
```

3. Read:

```
String value = et.getText().toString().trim();
```

4. Validation:

```
if (value.isEmpty()) {
et.setError("Required");
```

return;

}

5. Click:

btn.setOnClickListener(v -> { });

6. Toast:

Toast.makeText(this, "Saved",

Toast.LENGTH_SHORT).show();

7. Radio:

int id = rg.getCheckedRadioButtonId();

8. CheckBox:

if (cb.isChecked()) { }

9. Spinner:

spinner.setAdapter(adapter);

10. List:

listView.setAdapter(adapter);

11. Refresh:

adapter.notifyDataSetChanged();

12. Save:

sp.edit().putString("key", value).apply();

13. Read:

sp.getString("key", "default");

14. Alert:

new AlertDialog.Builder(this)

.setTitle("Confirm")

.setPositiveButton("Yes", null)

.setNegativeButton("No", null)

.show();

15. Explicit Intent:

Intent i = new Intent(this, SecondActivity.class);

i.putExtra("key", value);

startActivity(i);

16. Receive:

getIntent().getStringExtra("key");

17. Browser:

new Intent(Intent.ACTION_VIEW,

Uri.parse("https://www.google.com"));

18. Dial:

new Intent(Intent.ACTION_DIAL,

Uri.parse("tel:9876543210"));

19. SMS:

new Intent(Intent.ACTION_SENDTO,
Uri.parse("sms:9876543210"));

20. Share:

Intent.ACTION_SEND;

21. Dynamic UI:

container.addView(view);

22. Remove:

container.removeView(view);

23. Fragment:

getSupportFragmentManager()
.beginTransaction()
.replace(R.id.container, new HomeFragment())
.commit();

24. Back stack:

.addToBackStack(null)

25. Fragment data:

fragment.setArguments(bundle);
getArguments();

26. Context Menu:

registerForContextMenu(view);

27. Popup Menu:

new PopupMenu(this, anchor);

28. Options Menu:

onCreateOptionsMenu()
onOptionsItemSelected()

29. TextWatcher:

beforeTextChanged()
onTextChanged()
afterTextChanged()

30. Lifecycle:

onCreate
onStart
onResume
onPause
onStop
onRestart
onDestroy

THE LIFECYCLE MEMORY TRICK

CREATE → START → RESUME

↓

PAUSE

↓

STOP

↓

RESTART

↓

START → RESUME

FINISH / DESTROY:

PAUSE → STOP → DESTROY

THE MASTER EXAM MEMORY TRICK

INPUT

↓

VALIDATE

↓

STORE / CONFIRM

↓

SEND

↓

DISPLAY

↓

CLICK

↓

LONG CLICK

↓

ACTION

Final advice: learn the lifecycle and the reusable syntax patterns first. Once those are automatic, most of the model questions become changes to names, arrays, validation rules, messages, destinations and UI widgets rather than completely new programs.

PART 2 — LAMBDA + WITHOUT LAMBDA SIDE BY SIDE

Learn the lambda version for speed, but know the anonymous-class version because it is the expanded form and can be written when lambda syntax is forgotten.

Button click

LAMBDA	WITHOUT LAMBDA

`button.setOnClickListener(v -> {`
`  Toast.makeText(this, "Clicked",`
`  Toast.LENGTH_SHORT).show();`
`});`
	`button.setOnClickListener(`
	`  new View.OnClickListener() {`
	`  @Override`
	`  public void onClick(View v) {`
	`  Toast.makeText(`
	`  MainActivity.this,`
	`  "Clicked",`
	`  Toast.LENGTH_SHORT).show();`
	`  }`
	`  });`

ListView item click

LAMBDA	WITHOUT LAMBDA

`listView.setOnItemClickListener(`
`  (parent, view, position, id) -> {`
`  String item = items.get(position);`
`  });`
	`listView.setOnItemClickListener(`
	`  new AdapterView.OnItemClickListener() {`
	`  @Override`
	`  public void onItemClick(`
	`  AdapterView<?> parent,`
	`  View view,`
	`  int position,`
	`  long id) {`
	`  String item = items.get(position);`
	`  }`
	`  });`

CheckBox / Switch listener

LAMBDA	WITHOUT LAMBDA

`checkBox.setOnCheckedChangeListener(`
`  (buttonView, isChecked) -> {`
`  // logic`
`  });`
	`checkBox.setOnCheckedChangeListener(`
	`  new CompoundButton.OnCheckedChangeListener() {`
	`  @Override`
	`  public void onCheckedChanged(`
	`  CompoundButton buttonView,`
	`  boolean isChecked) {`
	`  // logic`
	`  }`
	`  });`

AlertDialog button

LAMBDA	WITHOUT LAMBDA

`.setPositiveButton("Yes",`
`  (dialog, which) -> save());`
	`.setPositiveButton(`
	`  "Yes",`
	`  new DialogInterface.OnClickListener() {`
	`  @Override`
	`  public void onClick(`
	`  DialogInterface dialog,`
	`  int which) {`
	`  save();`
	`  }`
	`  });`

DatePicker

LAMBDA	WITHOUT LAMBDA

`new DatePickerDialog(this,`
`  (view, year, month, day) -> {`
`  dateTextView.setText(`
`  day + "/" + (month + 1) + "/" + year);`
`  }, y, m, d).show();`
	`new DatePickerDialog(this,`
	`  new DatePickerDialog.OnDateSetListener() {`
	`  @Override`
	`  public void onDateSet(`
	`  DatePicker view, int year,`
	`  int month, int day) {`
	`  dateTextView.setText(`
	`  day + "/" + (month + 1) + "/" + year);`
	`  }`
	`  }, y, m, d).show();`

TimePicker

LAMBDA	WITHOUT LAMBDA

`new TimePickerDialog(this,`
`  (view, hour, minute) -> {`
`  timeTextView.setText(hour + ":" + minute);`
`  }, h, m, true).show();`
	`new TimePickerDialog(this,`
	`  new TimePickerDialog.OnTimeSetListener() {`
	`  @Override`
	`  public void onTimeSet(`
	`  TimePicker view,`
	`  int hour, int minute) {`
	`  timeTextView.setText(`
	`  hour + ":" + minute);`
	`  }`
	`  }, h, m, true).show();`

Editor action

LAMBDA	WITHOUT LAMBDA

editText.setOnEditorActionListener(
(v, actionId, event) -> {
if (actionId == EditorInfo.IME_ACTION_DONE) {
hideKeyboard(v);
return true;
}
return false;
});
	editText.setOnEditorActionListener(
	new TextView.OnEditorActionListener() {
	@Override
	public boolean onEditorAction(
	TextView v, int actionId,
	KeyEvent event) {
	if (actionId ==
	EditorInfo.IME_ACTION_DONE) {
	hideKeyboard(v);
	return true;
	}
	return false;
	}
	});

Memorize: lambda = shorter syntax; anonymous class = expanded syntax. If the exam permits either, use lambda for speed. If you forget a lambda, expand it into the anonymous-class listener.

PART 3 — SIMPLE PROGRAM FOR VERY LITTLE TIME

This is the small-time version. It deliberately covers the core master flow without every advanced widget.

SIMPLE MAD MASTER-PATTERN EXAMPLE

Student Registration → Home

This is a deliberately small example for understanding the pattern. It is not meant to demonstrate every feature from the big master program.

Pattern used: Activity 1 → form → validation → RadioButton + CheckBox → SharedPreferences → AlertDialog → Bundle → Explicit Intent → Activity 2 → ListView + ArrayAdapter → click → AlertDialog/Toast → long press → Context Menu → Implicit Intent.

Exam idea: Once you understand this program, replace Student Name / Course / Hobbies / list items / messages / Intent destination with whatever the question asks.

1. QUESTION FOR THIS EXAMPLE

Write an Android application to register a student. Take student name and course, select gender using RadioButton, select hobbies using CheckBoxes, validate the name, save the name using SharedPreferences, and move to a second Activity using an Explicit Intent. In the second Activity, display a ListView of subjects. On normal click show an AlertDialog and Toast. On long press show a Context Menu with Delete and Open Website.

This single small question lets you practice the reusable structure without making the program unnecessarily large.

2. PROJECT FILES

SimpleMAD/

└── app/src/main/

├── java/com/example/simplemad/

│ ├── MainActivity.java

│ └── HomeActivity.java

├── res/layout/

│ ├── activity_main.xml

│ └── activity_home.xml

├── res/menu/

│ └── context_menu.xml

└── AndroidManifest.xml

3. ACTIVITY 1 XML — activity_main.xml

Keep the XML simple. The important thing is to understand the IDs because Java uses these IDs with findViewById().

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent">
<LinearLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"
android:padding="20dp">
<!-- Student name -->
<EditText
android:id="@+id/nameEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Student Name"
android:inputType="textPersonName" />
```

```
<!-- Course -->
<EditText
android:id="@+id/courseEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Course"
android:inputType="text" />
<!-- Only one gender can be selected -->
<RadioGroup
android:id="@+id/genderRadioGroup"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal">
<RadioButton
android:id="@+id/maleRadioButton"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Male" />
<RadioButton
android:id="@+id/femaleRadioButton"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Female" />
</RadioGroup>
<!-- More than one hobby can be selected -->
<CheckBox
android:id="@+id/readingCheckBox"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Reading" />
<CheckBox
android:id="@+id/sportsCheckBox"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Sports" />
<Button
android:id="@+id/registerButton"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Register" />
<Button
android:id="@+id/showSavedButton"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Show Saved Name" />
</LinearLayout>
</ScrollView>
```

4. ACTIVITY 1 JAVA — MainActivity.java

Read this slowly. The comments show what each section is doing and why it exists.

```
package com.example.simplemad;
import android.app.AlertDialog;
import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.view.View;
import android.widget.*;
public class MainActivity extends Activity {
```

```
// Variables for the XML widgets
EditText nameEditText;
EditText courseEditText;
RadioGroup genderRadioGroup;
CheckBox readingCheckBox;
CheckBox sportsCheckBox;
Button registerButton;
Button showSavedButton;
// SharedPreferences stores small data permanently
SharedPreferences preferences;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
// Connect Java with activity_main.xml
setContentView(R.layout.activity_main);
// Find each XML view using its ID
nameEditText = findViewById(R.id.nameEditText);
courseEditText = findViewById(R.id.courseEditText);
genderRadioGroup = findViewById(R.id.genderRadioGroup);
readingCheckBox = findViewById(R.id.readingCheckBox);
sportsCheckBox = findViewById(R.id.sportsCheckBox);
registerButton = findViewById(R.id.registerButton);
showSavedButton = findViewById(R.id.showSavedButton);
// Create/open a SharedPreferences file
preferences = getSharedPreferences(
"StudentPrefs",
MODE_PRIVATE
);
// Register button click
registerButton.setOnClickListener(v -> registerStudent());
// Show saved name button click
showSavedButton.setOnClickListener(
v -> showSavedName()
);
}
// ------------------------------------------------
// VALIDATION + FORM READING + SHARED PREFERENCES
// ------------------------------------------------
private void registerStudent() {
// Read text from EditTexts
String name = nameEditText
.getText()
.toString()
.trim();
String course = courseEditText
.getText()
.toString()
.trim();
// Simple validation
if (name.isEmpty()) {
nameEditText.setError("Enter student name");
nameEditText.requestFocus();
return;
}
if (course.isEmpty()) {
courseEditText.setError("Enter course");
```

```
courseEditText.requestFocus();
return;
}
 // Read selected RadioButton
int selectedId =
genderRadioGroup.getCheckedRadioButtonId();
String gender = "Not selected";
if (selectedId != -1) {
RadioButton selectedRadioButton =
findViewById(selectedId);
gender = selectedRadioButton
.getText()
.toString();
}
 // Read CheckBox values
String hobbies = "";
if (readingCheckBox.isChecked()) {
hobbies += "Reading ";
}
if (sportsCheckBox.isChecked()) {
hobbies += "Sports";
}
if (hobbies.isEmpty()) {
hobbies = "None";
}
 // Save the name in SharedPreferences
preferences.edit()
.putString("studentName", name)
.apply();
 // AlertDialog gives a confirmation
new AlertDialog.Builder(this)
.setTitle("Registration")
.setMessage(
"Name: " + name +
"\nCourse: " + course +
"\nGender: " + gender +
"\nHobbies: " + hobbies
)
.setPositiveButton("Continue",
(dialog, which) -> openHome(name))
.setNegativeButton("Cancel", null)
.show();
}
 // ------------------------------------------------
 // EXPLICIT INTENT + BUNDLE
 // ------------------------------------------------
 private void openHome(String name) {
 // Explicit Intent means we directly specify
 // which Activity should open.
Intent intent = new Intent(
MainActivity.this,
HomeActivity.class
);
 // Bundle stores data to send to the next Activity
Bundle bundle = new Bundle();
```

```
bundle.putString("studentName", name);
 // Put Bundle inside Intent
intent.putExtras(bundle);
 // Start the second Activity
startActivity(intent);
}
 // ------------------------------------------------
 // READ SHARED PREFERENCES
 // ------------------------------------------------
 private void showSavedName() {
 // Read the saved value.
 // "No name saved" is the default value.
String savedName = preferences.getString(
"studentName",
"No name saved"
);
Toast.makeText(
this,
"Saved: " + savedName,
Toast.LENGTH_SHORT
).show();
}
}
```

5. WHAT TO UNDERSTAND FROM MainActivity

Code Concept

findViewById() Connects Java with XML views.

getText().toString().trim() Reads text from EditText.

setError() Shows validation error.

RadioGroup Allows one RadioButton choice.

isChecked() Checks a CheckBox.

SharedPreferences Stores small persistent data.

AlertDialog Shows confirmation/details.

Bundle Carries data between components.

Explicit Intent Opens a specific Activity.

6. ACTIVITY 2 XML — activity_home.xml

```
 <?xml version="1.0" encoding="utf-8"?>
 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
 <TextView
android:id="@+id/welcomeTextView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:textSize="22sp"
android:textStyle="bold" />
 <!-- ListView displays an array/list of subjects -->
 <ListView
android:id="@+id/subjectListView"
android:layout_width="match_parent"
android:layout_height="0dp"
android:layout_weight="1" />
 </LinearLayout>
```

7. ACTIVITY 2 JAVA — HomeActivity.java

```
package com.example.simplemad;
import android.app.*;
import android.content.*;
import android.net.Uri;
import android.os.Bundle;
import android.view.*;
import android.widget.*;
import java.util.ArrayList;
public class HomeActivity extends Activity {
TextView welcomeTextView;
ListView subjectListView;
ArrayList subjects;
ArrayAdapter adapter;
int selectedPosition = -1;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_home);
welcomeTextView =
findViewById(R.id.welcomeTextView);
subjectListView =
findViewById(R.id.subjectListView);
// ------------------------------------------
// READ DATA FROM EXPLICIT INTENT / BUNDLE
// ------------------------------------------
Bundle bundle = getIntent().getExtras();
if (bundle != null) {
String name = bundle.getString(
"studentName",
"Student"
);
welcomeTextView.setText(
"Welcome, " + name
);
}
// ------------------------------------------
// LISTVIEW + ARRAYADAPTER
// ------------------------------------------
subjects = new ArrayList<>();
subjects.add("Java");
subjects.add("Android");
subjects.add("XML");
subjects.add("Mobile Computing");
// Adapter connects the ArrayList to ListView
adapter = new ArrayAdapter<>(
this,
android.R.layout.simple_list_item_1,
subjects
);
subjectListView.setAdapter(adapter);
// ------------------------------------------
// NORMAL CLICK
// ------------------------------------------
subjectListView.setOnItemClickListener(
(parent, view, position, id) -> {
String subject =
subjects.get(position);
```

```
// Show selected subject
new AlertDialog.Builder(this)
.setTitle("Subject Selected")
.setMessage(subject)
.setPositiveButton("OK", null)
.show();
// Also show Toast
Toast.makeText(
this,
"You selected " + subject,
Toast.LENGTH_SHORT
).show();
}
);
// ------------------------------------------
// LONG CLICK
// ------------------------------------------
subjectListView.setOnItemLongClickListener(
(parent, view, position, id) -> {
// Remember which item was long pressed
selectedPosition = position;
// Show Context Menu
subjectListView.showContextMenu();
return true;
}
);
// Context menu is attached to the ListView
registerForContextMenu(subjectListView);
}
// ------------------------------------------
// CREATE CONTEXT MENU
// ------------------------------------------
@Override
public void onCreateContextMenu(
ContextMenu menu,
View v,
ContextMenu.ContextMenuInfo menuInfo) {
super.onCreateContextMenu(
menu,
v,
menuInfo
);
menu.setHeaderTitle("Choose Action");
menu.add("Delete");
menu.add("Open Website");
}
// ------------------------------------------
// CONTEXT MENU CLICK
// ------------------------------------------
@Override
public boolean onContextItemSelected(
MenuItem item) {
if (selectedPosition == -1) {
return super.onContextItemSelected(item);
}
String selectedSubject =
```

```
subjects.get(selectedPosition);
 // If Delete was selected
if (item.getTitle().toString()
.equals("Delete")) {
subjects.remove(selectedPosition);
 // Refresh ListView after deleting
adapter.notifyDataSetChanged();
Toast.makeText(
this,
"Deleted " + selectedSubject,
Toast.LENGTH_SHORT
).show();
selectedPosition = -1;
return true;
}
 // If Open Website was selected
if (item.getTitle().toString()
.equals("Open Website")) {
 // ---------------------------------------
 // IMPLICIT INTENT
 // ---------------------------------------
Intent intent = new Intent(
Intent.ACTION_VIEW,
Uri.parse("https://www.google.com")
);
 // Check whether an app can handle the Intent
if (intent.resolveActivity(
getPackageManager()) != null) {
startActivity(intent);
}
return true;
}
return super.onContextItemSelected(item);
}
}
```

8. MANIFEST

```
 <?xml version="1.0" encoding="utf-8"?>
 <manifest xmlns:android="http://schemas.android.com/apk/res/android">
 <application
android:label="Simple MAD">
 <!-- Second Activity -->
 <activity
android:name=".HomeActivity"
android:exported="false" />
 <!-- First Activity / Launcher -->
 <activity
android:name=".MainActivity"
android:exported="true">
 <intent-filter>
 <action android:name="android.intent.action.MAIN" />
 <category
android:name="android.intent.category.LAUNCHER" />
 </intent-filter>
 </activity>
 </application>
 </manifest>
```

9. THE SAME PROGRAM AS A MEMORY MAP

MAIN ACTIVITY

|

|-- findViewById()

|

|-- Read name/course

|

|-- Validate

| |

| +-- setError()

|

|-- RadioButton

|

|-- CheckBox

|

|-- SharedPreferences

|

|-- AlertDialog

|

|-- Bundle

|

+-- Explicit Intent

|

v

HOME ACTIVITY

|

|-- getIntent().getExtras()

|

|-- ListView

| |

| +-- ArrayAdapter

|

|-- Normal click

| |

| +-- AlertDialog + Toast

|

|-- Long click

| |

| +-- Context Menu

| |

| +-- Delete

| | |

| | +-- notifyDataSetChanged()

| |

| +-- Open Website

| |

| +-- Implicit Intent

|

v

END

10. HOW TO CHANGE THIS FOR ANY EXAM QUESTION

If question asks for... Change this example to...

Patient appointment name → patientName; course → appointmentType; add date/time.

Shopping cart subjects → products; CheckBoxes → products; calculate total.

Contact list subjects → contacts; Context Menu → Call/Edit/Delete.

Travel planner subjects → destinations; Implicit Intent → Maps.

Email utility Open Website → ACTION_SENDTO mailto.

Phone utility Open Website → ACTION_DIAL tel:.

SMS utility Open Website → ACTION_SENDTO sms:.

Calendar Use ACTION_INSERT for calendar event.

Fragments Add FrameLayout, create Fragment, replace(), addToBackStack(), setArguments()/getArguments().

11. MOST IMPORTANT LINES TO MEMORIZE

```
// 1. Read EditText
String name = nameEditText.getText().toString().trim();
// 2. Validate
if (name.isEmpty()) {
nameEditText.setError("Enter name");
return;
}
// 3. Toast
Toast.makeText(this, "Saved", Toast.LENGTH_SHORT).show();
// 4. Save preference
preferences.edit().putString("studentName", name).apply();
// 5. Read preference
String name = preferences.getString("studentName", "");
// 6. Explicit Intent
Intent intent = new Intent(MainActivity.this, HomeActivity.class);
startActivity(intent);
// 7. Send data
intent.putExtra("name", name);
// 8. Receive data
String name = getIntent().getStringExtra("name");
// 9. ListView adapter
ArrayAdapter adapter =
new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1,
subjects);
subjectListView.setAdapter(adapter);
// 10. Refresh after delete
adapter.notifyDataSetChanged();
// 11. Implicit Intent
Intent intent = new Intent(
Intent.ACTION_VIEW,
Uri.parse("https://www.google.com")
);
if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
```

12. EXAM RULE TO REMEMBER

Do not try to memorize 50 completely different programs. Memorize the pattern. Then change the names, arrays, messages and destination according to the question.

Think: Input → Validate → Store → Confirm → Send → Display → Click → Long Click → Action.

PART 4 — FINAL MASTER PROGRAM

The previous MasterMAD artifact already combines the Level 1/2/3 patterns into one reusable project. filecite turn18file2 L12-L25 This final edition keeps that material and adds a correction layer so you know what must be used in the final implementation.

MASTER PROJECT FILES

```
MasterMAD/
└── app/src/main/
    ├── java/com/example/mastermad/
    │   ├── MainActivity.java
    │   ├── DashboardActivity.java
    │   ├── HomeFragment.java
    │   └── DetailsFragment.java
    ├── res/layout/
    │   ├── activity_main.xml
    │   ├── activity_dashboard.xml
    │   ├── fragment_home.xml
    │   ├── fragment_details.xml
    │   └── dialog_add_item.xml
    ├── res/menu/
    │   ├── menu_main.xml
    │   └── menu_dashboard.xml
    └── AndroidManifest.xml
```

MIN SDK + JAVA/XML BASELINE

```
minSdk 23

Use:
Java + XML
AndroidX
AppCompatActivity
androidx.fragment.app.Fragment
findViewById()

Do not add SQLite/Room because they are outside
the requested non-SQLite question banks.
```

FINAL MASTER CODE — IMPORTANT CORRECTIONS

Use AppCompatActivity consistently in Activities. Use androidx.fragment.app.Fragment consistently in Fragments. Keep all FragmentManager operations in the AndroidX support FragmentManager. Keep resolveActivity() checks for external Intents. The older artifact contains the complete teaching code; the following correction block is the final rule set.

```
// Activity
public class MainActivity extends AppCompatActivity { }

// Fragment
public class HomeFragment extends Fragment { }

// Fragment transaction
getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.fragmentContainer,
             new HomeFragment())
    .addToBackStack(null)
    .commit();

// External Intent
Intent intent = new Intent(
    Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com"));

if (intent.resolveActivity(
        getPackageManager()) != null) {
    startActivity(intent);
} else {
    Toast.makeText(this,
        "No suitable app found",
        Toast.LENGTH_SHORT).show();
}
```

FULL MASTER JAVA/XML MATERIAL FROM THE PREVIOUS MASTER EDITION

The complete previous MasterMAD code, comments, layouts, fragments, menus and manifest are retained below as the core master reference. filecite turn18file2 L46-L49

MASTER MAD — ANDROID JAVA + XML EXAM

PATTERN

Reusable-pattern program for Mobile Application Development Lab

Goal: Learn one reusable master structure and then change the field names, arrays, messages and Intent

destination according to the exam question.

Technology: Java + XML. Beginner-friendly Android programming. No Kotlin, Compose or SQLite.

Memorization flow: Form → Validation → Widgets → Preferences → Dialog → Bundle → Explicit Intent → Second

Activity → ListView → Adapter → Click → AlertDialog/Toast → Long Click → Context Menu → Implicit Intent →

Fragment → Bundle → Dynamic UI.

1. WHAT YOU MUST MASTER

LEVEL 1 — MUST KNOW PERFECTLY

Login, Calculator, Explicit Intent, Activity Lifecycle, RadioButton + CheckBox, Implicit Intent all actions, ListView + ArrayAdapter, ScrollView + dynamic addView, Spinner + Adapter + OnItemSelectedListener, `SharedPreferences`.

LEVEL 2 — VERY IMPORTANT

AlertDialog, PopupMenu, Options Menu, Context Menu, Custom Dialog, Long-click, notifyDataSetChanged, TextWatcher, RatingBar, DatePickerDialog, TimePickerDialog, IME Done, Fragments, FragmentManager, replace, addToBackStack, setArguments, getArguments, Bundle, dynamic UI, ProgressBar, AutoCompleteTextView, TableLayout, GridLayout.

LEVEL 3 — EXAM MASTER PATTERN

Activity1 → Form widgets → Validation → Options Menu/Button → AlertDialog → SharedPreferences → Bundle → Explicit Intent → Activity2 → Read preferences/Intent data → ListView + Adapter → Normal click → AlertDialog/Toast → Long press → Context Menu → Implicit Intent Browser/Dialer/Email/Maps/Calendar.

2. PROJECT STRUCTURE

```
MasterMAD/
├── app/
│ └── src/main/
│ ├── java/com/example/mastermad/
│ │ ├── MainActivity.java
│ │ ├── DashboardActivity.java
│ │ ├── HomeFragment.java
│ │ └── DetailsFragment.java
│ ├── res/
│ │ ├── layout/
│ │ │ ├── activity_main.xml
│ │ │ ├── activity_dashboard.xml
│ │ │ ├── fragment_home.xml
│ │ │ ├── fragment_details.xml
│ │ │ └── dialog_add_item.xml
│ │ └── menu/
│ │ ├── menu_main.xml
│ │ └── menu_dashboard.xml
│ └── AndroidManifest.xml
```

3. MASTER JAVA — ACTIVITY 1

This activity combines the main reusable patterns: form widgets, validation, SharedPreferences, Spinner, AutoCompleteTextView, TextWatcher, IME Done, date/time dialogs, RatingBar, calculator, dynamic addView, Options Menu, AlertDialog, custom dialog, Bundle, Explicit Intent and lifecycle.

```
package com.example.mastermad;
import android.app.*;
import android.content.*;
import android.os.Bundle;
import android.text.*;
import android.text.method.PasswordTransformationMethod;
import android.util.Log;
import android.view.*;
import android.view.inputmethod.EditorInfo;
import android.widget.*;
import java.text.SimpleDateFormat;
import java.util.*;
public class MainActivity extends Activity {
private EditText nameEditText, emailEditText, passwordEditText;
private EditText numberOneEditText, numberTwoEditText, commentEditText;
private TextView resultTextView, dateTextView, timeTextView, dynamicTextView;
```

```
 private RadioGroup genderRadioGroup;
 private CheckBox readingCheckBox, sportsCheckBox;
 private Spinner courseSpinner;
 private AutoCompleteTextView cityAutoCompleteTextView;
 private RatingBar ratingBar;
 private LinearLayout dynamicContainer;
 private SharedPreferences sharedPreferences;
 private final String[] courses = {
"MCA", "MBA", "MSc Computer Science", "BCA"
};
 private final String[] cities = {
"Thiruvananthapuram", "Kochi", "Kollam",
"Kozhikode", "Kannur"
};
 @Override
 protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
 // Form widgets
nameEditText = findViewById(R.id.nameEditText);
emailEditText = findViewById(R.id.emailEditText);
passwordEditText = findViewById(R.id.passwordEditText);
numberOneEditText = findViewById(R.id.numberOneEditText);
numberTwoEditText = findViewById(R.id.numberTwoEditText);
commentEditText = findViewById(R.id.commentEditText);
resultTextView = findViewById(R.id.resultTextView);
dateTextView = findViewById(R.id.dateTextView);
timeTextView = findViewById(R.id.timeTextView);
dynamicTextView = findViewById(R.id.dynamicTextView);
genderRadioGroup = findViewById(R.id.genderRadioGroup);
readingCheckBox = findViewById(R.id.readingCheckBox);
sportsCheckBox = findViewById(R.id.sportsCheckBox);
courseSpinner = findViewById(R.id.courseSpinner);
cityAutoCompleteTextView = findViewById(R.id.cityAutoCompleteTextView);
ratingBar = findViewById(R.id.ratingBar);
dynamicContainer = findViewById(R.id.dynamicContainer);
sharedPreferences = getSharedPreferences("MasterPrefs", MODE_PRIVATE);
setupSpinner();
setupAutoComplete();
setupTextWatcher();
setupImeDone();
findViewById(R.id.saveButton).setOnClickListener(v -> saveUser());
findViewById(R.id.showSavedButton).setOnClickListener(v -> showSavedUser());
findViewById(R.id.loginButton).setOnClickListener(v -> openDashboard());
findViewById(R.id.addDynamicButton).setOnClickListener(v -> addDynamicView());
findViewById(R.id.addButton).setOnClickListener(v -> calculate("+"));
findViewById(R.id.subtractButton).setOnClickListener(v -> calculate("-"));
findViewById(R.id.multiplyButton).setOnClickListener(v -> calculate("*"));
findViewById(R.id.divideButton).setOnClickListener(v -> calculate("/"));
findViewById(R.id.clearButton).setOnClickListener(v -> resultTextView.setText(""));
findViewById(R.id.dateButton).setOnClickListener(v -> showDatePicker());
findViewById(R.id.timeButton).setOnClickListener(v -> showTimePicker());
findViewById(R.id.alertButton).setOnClickListener(v -> showAlertDialog());
findViewById(R.id.customDialogButton).setOnClickListener(v -> showCustomDialog());
}
 // ---------------- VALIDATION + EXPLICIT INTENT ----------------
```

```
 private boolean validateForm() {
String name = nameEditText.getText().toString().trim();
String email = emailEditText.getText().toString().trim();
String password = passwordEditText.getText().toString().trim();
 if (name.isEmpty()) {
nameEditText.setError("Enter name");
nameEditText.requestFocus();
 return false;
}
 if (email.isEmpty()) {
emailEditText.setError("Enter email");
emailEditText.requestFocus();
 return false;
}
 if (password.isEmpty()) {
passwordEditText.setError("Enter password");
passwordEditText.requestFocus();
 return false;
}
 if (password.length() < 6) {
passwordEditText.setError("Minimum 6 characters");
passwordEditText.requestFocus();
 return false;
}
 return true;
}
 private void openDashboard() {
 if (!validateForm()) return;
String name = nameEditText.getText().toString().trim();
String email = emailEditText.getText().toString().trim();
 Intent intent = new Intent(MainActivity.this, DashboardActivity.class);
 Bundle bundle = new Bundle();
bundle.putString("name", name);
bundle.putString("email", email);
intent.putExtras(bundle);
startActivity(intent);
}
 // ---------------- SHARED PREFERENCES ----------------
 private void saveUser() {
 if (!validateForm()) return;
String name = nameEditText.getText().toString().trim();
String email = emailEditText.getText().toString().trim();
 SharedPreferences.Editor editor = sharedPreferences.edit();
editor.putString("name", name);
editor.putString("email", email);
editor.apply();
 Toast.makeText(this, "Data saved", Toast.LENGTH_SHORT).show();
}
 private void showSavedUser() {
String name = sharedPreferences.getString("name", "No name");
String email = sharedPreferences.getString("email", "No email");
 new AlertDialog.Builder(this)
.setTitle("Saved Details")
.setMessage("Name: " + name + "\nEmail: " + email)
.setPositiveButton("OK", null)
.show();
}
```

```
// ---------------- SPINNER ----------------
private void setupSpinner() {
ArrayAdapter<String> adapter = new ArrayAdapter<>(
this,
android.R.layout.simple_spinner_item,
courses
);
adapter.setDropDownViewResource(
android.R.layout.simple_spinner_dropdown_item
);
courseSpinner.setAdapter(adapter);
courseSpinner.setOnItemSelectedListener(
new AdapterView.OnItemSelectedListener() {
@Override
public void onItemSelected(
AdapterView parent, View view,
int position, long id) {
Toast.makeText(
MainActivity.this,
"Selected: " + courses[position],
Toast.LENGTH_SHORT
).show();
}
@Override
public void onNothingSelected(AdapterView parent) {
}
}
);
}
// ---------------- AUTOCOMPLETE ----------------
private void setupAutoComplete() {
ArrayAdapter<String> adapter = new ArrayAdapter<>(
this,
android.R.layout.simple_dropdown_item_1line,
cities
);
cityAutoCompleteTextView.setAdapter(adapter);
cityAutoCompleteTextView.setThreshold(1);
}
// ---------------- TEXTWATCHER ----------------
private void setupTextWatcher() {
commentEditText.addTextChangedListener(
new TextWatcher() {
@Override
public void beforeTextChanged(
CharSequence s, int start,
int count, int after) {
}
@Override
public void onTextChanged(
CharSequence s, int start,
int before, int count) {
dynamicTextView.setText(
"Characters: " + s.length()
);
}
```

```
@Override
public void afterTextChanged(Editable s) {
}
}
);
}
// ---------------- IME DONE ----------------
private void setupImeDone() {
commentEditText.setOnEditorActionListener(
(v, actionId, event) -> {
if (actionId == EditorInfo.IME_ACTION_DONE) {
v.clearFocus();
return true;
}
return false;
}
);
}
// ---------------- DATE PICKER ----------------
private void showDatePicker() {
Calendar calendar = Calendar.getInstance();
DatePickerDialog dialog = new DatePickerDialog(
this,
(view, year, month, dayOfMonth) -> {
String date = dayOfMonth + "/" +
(month + 1) + "/" + year;
dateTextView.setText("Date: " + date);
},
calendar.get(Calendar.YEAR),
calendar.get(Calendar.MONTH),
calendar.get(Calendar.DAY_OF_MONTH)
);
dialog.getDatePicker().setMinDate(
System.currentTimeMillis()
);
dialog.show();
}
// ---------------- TIME PICKER ----------------
private void showTimePicker() {
Calendar calendar = Calendar.getInstance();
TimePickerDialog dialog = new TimePickerDialog(
this,
(view, hourOfDay, minute) -> {
String time = String.format(
Locale.getDefault(),
"%02d:%02d",
hourOfDay, minute
);
timeTextView.setText("Time: " + time);
},
calendar.get(Calendar.HOUR_OF_DAY),
calendar.get(Calendar.MINUTE),
true
);
dialog.show();
```

```
}
 // ---------------- RATING BAR ----------------
 private float getRating() {
 return ratingBar.getRating();
}
 // ---------------- CALCULATOR ----------------
 private void calculate(String operator) {
try {
double first = Double.parseDouble(
numberOneEditText.getText().toString().trim()
);
double second = Double.parseDouble(
numberTwoEditText.getText().toString().trim()
);
double answer;
switch (operator) {
case "+":
answer = first + second;
break;
case "-":
answer = first - second;
break;
case "*":
answer = first * second;
break;
case "/":
 if (second == 0) {
resultTextView.setText("Cannot divide by zero");
return;
}
answer = first / second;
break;
default:
return;
}
resultTextView.setText(String.valueOf(answer));
} catch (NumberFormatException e) {
resultTextView.setText("Enter valid numbers");
}
}
 // ---------------- DYNAMIC UI ----------------
 private void addDynamicView() {
TextView textView = new TextView(this);
textView.setText("Dynamic item " +
(dynamicContainer.getChildCount() + 1));
textView.setTextSize(16);
textView.setPadding(12, 12, 12, 12);
dynamicContainer.addView(textView);
}
 // ---------------- ALERT DIALOG ----------------
 private void showAlertDialog() {
 new AlertDialog.Builder(this)
.setTitle("Confirmation")
.setMessage("Do you want to continue?")
.setPositiveButton("Yes", (dialog, which) ->
 Toast.makeText(
```

```
this,
"Yes selected",
 Toast.LENGTH_SHORT
).show()
)
.setNegativeButton("No", null)
.show();
}
 // ---------------- CUSTOM DIALOG ----------------
 private void showCustomDialog() {
View view = getLayoutInflater().inflate(
R.layout.dialog_add_item, null
);
EditText itemNameEditText =
view.findViewById(R.id.itemNameEditText);
EditText quantityEditText =
view.findViewById(R.id.quantityEditText);
 new AlertDialog.Builder(this)
.setTitle("Add Item")
.setView(view)
.setPositiveButton("Save", (dialog, which) -> {
String itemName =
itemNameEditText.getText().toString().trim();
String quantity =
quantityEditText.getText().toString().trim();
 if (itemName.isEmpty() ||
quantity.isEmpty()) {
 Toast.makeText(
this,
"Enter all details",
 Toast.LENGTH_SHORT
).show();
} else {
 Toast.makeText(
this,
"Item added",
 Toast.LENGTH_SHORT
).show();
}
})
.setNegativeButton("Cancel", null)
.show();
}
 // ---------------- OPTIONS MENU ----------------
 @Override
public boolean onCreateOptionsMenu(Menu menu) {
getMenuInflater().inflate(
R.menu.menu_main,
menu
);
 return true;
}
 @Override
public boolean onOptionsItemSelected(MenuItem item) {
 if (item.getItemId() == R.id.menu_save) {
saveUser();
```

```
 return true;
}
 if (item.getItemId() == R.id.menu_about) {
showAlertDialog();
 return true;
}
 return super.onOptionsItemSelected(item);
}
 // ---------------- ACTIVITY LIFECYCLE ----------------
 @Override
 protected void onStart() {
super.onStart();
Log.d("LIFECYCLE", "onStart");
}
 @Override
 protected void onResume() {
super.onResume();
Log.d("LIFECYCLE", "onResume");
}
 @Override
 protected void onPause() {
super.onPause();
Log.d("LIFECYCLE", "onPause");
}
 @Override
 protected void onStop() {
super.onStop();
Log.d("LIFECYCLE", "onStop");
}
 @Override
 protected void onRestart() {
super.onRestart();
Log.d("LIFECYCLE", "onRestart");
}
 @Override
 protected void onDestroy() {
super.onDestroy();
Log.d("LIFECYCLE", "onDestroy");
}
}
```

4. MASTER XML — ACTIVITY 1

The layout deliberately demonstrates ScrollView, LinearLayout, TableLayout, GridLayout, EditText, RadioGroup/RadioButton, CheckBox, Spinner, AutoCompleteTextView, RatingBar, buttons and a dynamic container.

```
 <?xml version="1.0" encoding="utf-8"?>
 <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent">
 <LinearLayout
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="vertical"
 android:padding="16dp">
 <TextView
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Master MAD Form"
 android:textSize="24sp"
```

```
android:textStyle="bold" />
<TableLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:stretchColumns="1">
<TableRow>
<TextView
android:text="Name"
android:padding="8dp" />
<EditText
android:id="@+id/nameEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter name"
android:inputType="textPersonName" />
</TableRow>
<TableRow>
<TextView
android:text="Email"
android:padding="8dp" />
<EditText
android:id="@+id/emailEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter email"
android:inputType="textEmailAddress" />
</TableRow>
<TableRow>
<TextView
android:text="Password"
android:padding="8dp" />
<EditText
android:id="@+id/passwordEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter password"
android:inputType="textPassword" />
</TableRow>
</TableLayout>
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Gender" />
<RadioGroup
android:id="@+id/genderRadioGroup"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal">
<RadioButton
android:id="@+id/maleRadioButton"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Male" />
<RadioButton
android:id="@+id/femaleRadioButton"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Female" />
</RadioGroup>
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hobbies" />
<CheckBox
```

```
android:id="@+id/readingCheckBox"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Reading" />
<CheckBox
android:id="@+id/sportsCheckBox"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Sports" />
<Spinner
android:id="@+id/courseSpinner"
android:layout_width="match_parent"
android:layout_height="wrap_content" />
<AutoCompleteTextView
android:id="@+id/cityAutoCompleteTextView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="City"
android:inputType="text" />
<RatingBar
android:id="@+id/ratingBar"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:numStars="5"
android:stepSize="1" />
<EditText
android:id="@+id/commentEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Comment"
android:inputType="textMultiLine"
android:imeOptions="actionDone"
android:minLines="3" />
<TextView
android:id="@+id/dynamicTextView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Characters: 0" />
<GridLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:columnCount="2">
<Button
android:id="@+id/saveButton"
android:text="Save" />
<Button
android:id="@+id/showSavedButton"
android:text="Show Saved" />
<Button
android:id="@+id/loginButton"
android:text="Open Dashboard" />
<Button
android:id="@+id/alertButton"
android:text="AlertDialog" />
<Button
android:id="@+id/customDialogButton"
android:text="Custom Dialog" />
<Button
android:id="@+id/addDynamicButton"
android:text="Add Dynamic View" />
</GridLayout>
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
```

```
android:text="Date and Time" />
<Button
android:id="@+id/dateButton"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Select Date" />
<TextView
android:id="@+id/dateTextView"
android:layout_width="wrap_content"
android:layout_height="wrap_content" />
<Button
android:id="@+id/timeButton"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Select Time" />
<TextView
android:id="@+id/timeTextView"
android:layout_width="wrap_content"
android:layout_height="wrap_content" />
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Calculator"
android:textStyle="bold" />
<EditText
android:id="@+id/numberOneEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="First number"
android:inputType="numberDecimal" />
<EditText
android:id="@+id/numberTwoEditText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Second number"
android:inputType="numberDecimal" />
<GridLayout
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:columnCount="4">
<Button
android:id="@+id/addButton"
android:text="+" />
<Button
android:id="@+id/subtractButton"
android:text="-" />
<Button
android:id="@+id/multiplyButton"
android:text="*" />
<Button
android:id="@+id/divideButton"
android:text="/" />
</GridLayout>
<Button
android:id="@+id/clearButton"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Clear" />
<TextView
android:id="@+id/resultTextView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:textSize="20sp" />
<LinearLayout
```

```
android:id="@+id/dynamicContainer"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical" />
</LinearLayout>
</ScrollView>
```

5. MASTER JAVA — ACTIVITY 2 / DASHBOARD

This activity demonstrates reading Intent data, reading SharedPreferences, ListView + ArrayAdapter, normal click, AlertDialog, Toast, long-click, Context Menu, Implicit Intents, ProgressBar, fragments, FragmentManager, replace, addToBackStack and PopupMenu.

```
package com.example.mastermad;
import android.app.*;
import android.content.*;
import android.net.Uri;
import android.os.Bundle;
import android.view.*;
import android.widget.*;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;
import java.util.*;
public class DashboardActivity extends Activity {
private ListView itemListView;
private ArrayAdapter<String> adapter;
private ArrayList<String> items;
private ProgressBar progressBar;
private SharedPreferences sharedPreferences;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_dashboard);
TextView welcomeTextView = findViewById(R.id.welcomeTextView);
TextView detailsTextView = findViewById(R.id.detailsTextView);
itemListView = findViewById(R.id.itemListView);
progressBar = findViewById(R.id.progressBar);
sharedPreferences =
getSharedPreferences("MasterPrefs", MODE_PRIVATE);
// Read Intent / Bundle data
Bundle bundle = getIntent().getExtras();
if (bundle != null) {
String name = bundle.getString("name", "User");
String email = bundle.getString("email", "No email");
welcomeTextView.setText("Welcome, " + name);
detailsTextView.setText("Email: " + email);
}
// Read SharedPreferences
String savedName =
sharedPreferences.getString("name", "User");
if (bundle == null) {
welcomeTextView.setText("Welcome, " + savedName);
}
// ProgressBar
progressBar.setProgress(70);
// ListView + ArrayAdapter
items = new ArrayList<>();
items.add("Android");
items.add("Java");
items.add("XML");
```

```
items.add("Database");
items.add("Mobile Application");
adapter = new ArrayAdapter<>(
this,
android.R.layout.simple_list_item_1,
items
);
itemListView.setAdapter(adapter);
 // Normal click
itemListView.setOnItemClickListener(
(parent, view, position, id) -> {
String selectedItem = items.get(position);
 new AlertDialog.Builder(this)
.setTitle("Selected Item")
.setMessage(selectedItem)
.setPositiveButton("OK", null)
.show();
 Toast.makeText(
this,
"Clicked: " + selectedItem,
 Toast.LENGTH_SHORT
).show();
}
);
 // Long click
itemListView.setOnItemLongClickListener(
(parent, view, position, id) -> {
selectedPosition = position;
itemListView.showContextMenu();
 return true;
}
);
registerForContextMenu(itemListView);
findViewById(R.id.homeFragmentButton)
.setOnClickListener(v ->
openFragment(new HomeFragment(), false)
);
findViewById(R.id.detailsFragmentButton)
.setOnClickListener(v -> {
DetailsFragment fragment =
 new DetailsFragment();
 Bundle fragmentBundle = new Bundle();
fragmentBundle.putString(
"message",
"Data sent from Activity"
);
fragment.setArguments(fragmentBundle);
openFragment(fragment, true);
});
findViewById(R.id.popupButton)
.setOnClickListener(v -> showPopupMenu(v));
}
 private int selectedPosition = -1;
 // ---------------- CONTEXT MENU ----------------
 @Override
```

```
public void onCreateContextMenu(
ContextMenu menu,
View v,
ContextMenu.ContextMenuInfo menuInfo) {
super.onCreateContextMenu(menu, v, menuInfo);
menu.setHeaderTitle("Choose Action");
menu.add(0, 1, 0, "Delete");
menu.add(0, 2, 0, "Open Website");
menu.add(0, 3, 0, "Call");
menu.add(0, 4, 0, "SMS");
menu.add(0, 5, 0, "Share");
}
 @Override
public boolean onContextItemSelected(MenuItem item) {
 if (selectedPosition < 0 ||
selectedPosition >= items.size()) {
 return super.onContextItemSelected(item);
}
String selectedItem = items.get(selectedPosition);
switch (item.getItemId()) {
case 1:
items.remove(selectedPosition);
adapter.notifyDataSetChanged();
 Toast.makeText(
this,
"Deleted",
 Toast.LENGTH_SHORT
).show();
selectedPosition = -1;
 return true;
case 2:
openBrowser("https://www.google.com");
 return true;
case 3:
openDialer("tel:9876543210");
 return true;
case 4:
openSms("sms:9876543210");
 return true;
case 5:
shareText(selectedItem);
 return true;
}
 return super.onContextItemSelected(item);
}
 // ---------------- IMPLICIT INTENTS ----------------
 private void openBrowser(String url) {
 Intent intent = new Intent(
Intent.ACTION_VIEW,
Uri.parse(url)
);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
}
 private void openDialer(String phoneNumber) {
```

```
 Intent intent = new Intent(
Intent.ACTION_DIAL,
Uri.parse(phoneNumber)
);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
}
 private void openSms(String phoneNumber) {
 Intent intent = new Intent(
Intent.ACTION_SENDTO,
Uri.parse(phoneNumber)
);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
}
 private void sendEmail() {
 Intent intent = new Intent(Intent.ACTION_SENDTO);
intent.setData(Uri.parse(
"mailto:example@gmail.com"
));
intent.putExtra(
Intent.EXTRA_SUBJECT,
"MAD Exam"
);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
}
 private void openMap() {
Uri location = Uri.parse(
"geo:0,0?q=Thiruvananthapuram"
);
 Intent intent = new Intent(
Intent.ACTION_VIEW,
location
);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
}
 private void openCalendar() {
 Intent intent = new Intent(
Intent.ACTION_INSERT
);
intent.setData(
Uri.parse("content://com.android.calendar/events")
);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(intent);
}
}
 private void shareText(String text) {
 Intent intent = new Intent(Intent.ACTION_SEND);
intent.setType("text/plain");
```

```
intent.putExtra(Intent.EXTRA_TEXT, text);
 if (intent.resolveActivity(getPackageManager()) != null) {
startActivity(Intent.createChooser(
intent,
"Share using"
));
}
}
 // ---------------- FRAGMENTS ----------------
 private void openFragment(
Fragment fragment,
boolean addToBackStack) {
FragmentManager manager =
getFragmentManager();
FragmentTransaction transaction =
manager.beginTransaction();
transaction.replace(
R.id.fragmentContainer,
fragment
);
 if (addToBackStack) {
transaction.addToBackStack(null);
}
transaction.commit();
}
 // ---------------- POPUP MENU ----------------
 private void showPopupMenu(View anchor) {
PopupMenu popupMenu =
 new PopupMenu(this, anchor);
popupMenu.getMenu().add("Edit");
popupMenu.getMenu().add("Delete");
popupMenu.getMenu().add("Share");
popupMenu.setOnMenuItemClickListener(
item -> {
 Toast.makeText(
this,
item.getTitle() + " selected",
 Toast.LENGTH_SHORT
).show();
 return true;
}
);
popupMenu.show();
}
 // ---------------- OPTIONS MENU ----------------
 @Override
public boolean onCreateOptionsMenu(Menu menu) {
getMenuInflater().inflate(
R.menu.menu_dashboard,
menu
);
 return true;
}
 @Override
public boolean onOptionsItemSelected(
MenuItem item) {
```

```
 if (item.getItemId() == R.id.menu_settings) {
 Toast.makeText(
this,
"Settings",
 Toast.LENGTH_SHORT
).show();
 return true;
}
 return super.onOptionsItemSelected(item);
}
}
```

Important implementation note:

If using the platform android.app.Fragment classes above, the Fragment types and FragmentManager imports must remain from android.app. If using AndroidX instead, change the imports consistently to androidx.fragment.app.* and use androidx.appcompat.app.AppCompatActivity. Do not mix the two Fragment APIs.

6. DASHBOARD XML

```
 <?xml version="1.0" encoding="utf-8"?>
 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:orientation="vertical"
 android:padding="16dp">
 <TextView
 android:id="@+id/welcomeTextView"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:text="Welcome"
 android:textSize="24sp"
 android:textStyle="bold" />
 <TextView
 android:id="@+id/detailsTextView"
 android:layout_width="match_parent"
 android:layout_height="wrap_content" />
 <ProgressBar
 android:id="@+id/progressBar"
style="?android:attr/progressBarStyleHorizontal"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:max="100"
 android:progress="0" />
 <ListView
 android:id="@+id/itemListView"
 android:layout_width="match_parent"
 android:layout_height="0dp"
 android:layout_weight="1" />
 <LinearLayout
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="horizontal">
 <Button
 android:id="@+id/homeFragmentButton"
 android:layout_width="0dp"
 android:layout_height="wrap_content"
 android:layout_weight="1"
 android:text="Home" />
 <Button
 android:id="@+id/detailsFragmentButton"
 android:layout_width="0dp"
 android:layout_height="wrap_content"
 android:layout_weight="1"
 android:text="Details" />
```

```
<Button
android:id="@+id/popupButton"
android:layout_width="0dp"
android:layout_height="wrap_content"
android:layout_weight="1"
android:text="Popup" />
</LinearLayout>
<FrameLayout
android:id="@+id/fragmentContainer"
android:layout_width="match_parent"
android:layout_height="180dp" />
</LinearLayout>
```

7. HOME FRAGMENT

```
package com.example.mastermad;
import android.app.Fragment;
import android.os.Bundle;
import android.view.*;
import android.widget.*;
public class HomeFragment extends Fragment {
@Override
public View onCreateView(
LayoutInflater inflater,
ViewGroup container,
Bundle savedInstanceState) {
return inflater.inflate(
R.layout.fragment_home,
container,
false
);
}
}
```

fragment_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="16dp">
<TextView
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Home Fragment"
android:textSize="20sp"
android:textStyle="bold" />
</LinearLayout>
```

8. DETAILS FRAGMENT — setArguments / getArguments

```
package com.example.mastermad;
import android.app.Fragment;
import android.os.Bundle;
import android.view.*;
import android.widget.TextView;
public class DetailsFragment extends Fragment {
@Override
public View onCreateView(
LayoutInflater inflater,
ViewGroup container,
Bundle savedInstanceState) {
View view = inflater.inflate(
R.layout.fragment_details,
```

```
container,
false
);
TextView messageTextView =
view.findViewById(R.id.messageTextView);
 Bundle bundle = getArguments();
 if (bundle != null) {
String message =
bundle.getString(
"message",
"No data"
);
messageTextView.setText(message);
}
 return view;
}
}
```

fragment_details.xml

```
 <?xml version="1.0" encoding="utf-8"?>
 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:orientation="vertical"
 android:padding="16dp">
 <TextView
 android:id="@+id/messageTextView"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:textSize="18sp" />
 </LinearLayout>
```

9. CUSTOM DIALOG XML

```
 <?xml version="1.0" encoding="utf-8"?>
 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="vertical"
 android:padding="20dp">
 <EditText
 android:id="@+id/itemNameEditText"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:hint="Item name"
 android:inputType="text" />
 <EditText
 android:id="@+id/quantityEditText"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:hint="Quantity"
 android:inputType="number" />
 </LinearLayout>
```

10. OPTIONS MENU XML

```
 <?xml version="1.0" encoding="utf-8"?>
 <menu xmlns:android="http://schemas.android.com/apk/res/android">
 <item
 android:id="@+id/menu_save"
 android:title="Save" />
 <item
 android:id="@+id/menu_about"
 android:title="About" />
 </menu>
```

11. DASHBOARD MENU XML

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
<item
android:id="@+id/menu_settings"
android:title="Settings" />
</menu>
```

12. MANIFEST

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
<application
android:theme="@style/Theme.AppCompat.Light.NoActionBar"
android:label="Master MAD">
<activity
android:name=".DashboardActivity"
android:exported="false" />
<activity
android:name=".MainActivity"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
</application>
</manifest>
```

13. THE 10 CORE SNIPPETS TO MEMORIZE

These are the small patterns you should be able to reproduce even if the question changes completely.

1. findViewById

```
EditText nameEditText = findViewById(R.id.nameEditText);
```

2. Read text

```
String name = nameEditText.getText().toString().trim();
```

3. Button click

```
button.setOnClickListener(v -> {
// code
});
```

4. Validation

```
if (name.isEmpty()) {
nameEditText.setError("Enter name");
nameEditText.requestFocus();
return;
}
```

5. Toast

```
Toast.makeText(this, "Saved", Toast.LENGTH_SHORT).show();
```

6. Explicit Intent + Bundle

```
Intent intent = new Intent(MainActivity.this, DashboardActivity.class);
Bundle bundle = new Bundle();
bundle.putString("name", name);
intent.putExtras(bundle);
startActivity(intent);
```

7. SharedPreferences

```
SharedPreferences preferences =
getSharedPreferences("MyPrefs", MODE_PRIVATE);
preferences.edit()
.putString("name", name)
```

.apply();

String savedName = preferences.getString("name", "");

8. ListView + ArrayAdapter

```
ArrayAdapter<String> adapter = new ArrayAdapter<>(
```

this,

android.R.layout.simple_list_item_1,

items

);

listView.setAdapter(adapter);

9. Implicit Intent + resolveActivity

```
Intent intent = new Intent(Intent.ACTION_VIEW,
```

Uri.parse("https://www.google.com"));

```
if (intent.resolveActivity(getPackageManager()) != null) {
```

startActivity(intent);

}

10. Fragment data transfer

```
Bundle bundle = new Bundle();
```

bundle.putString("message", "Hello");

fragment.setArguments(bundle);

```
// inside Fragment
Bundle data = getArguments();
```

String message = data.getString("message");

14. IMPLICIT INTENT ACTIONS — EXAM TABLE

Purpose Action / Pattern Example

Browser / Website ACTION_VIEW Uri.parse("https://www.google.com")

Dialer ACTION_DIAL Uri.parse("tel:9876543210")

SMS ACTION_SENDTO Uri.parse("sms:9876543210")

Email ACTION_SENDTO Uri.parse("mailto:example@gmail.com")

Maps ACTION_VIEW Uri.parse("geo:0,0?q=Kochi")

Calendar ACTION_INSERT content://com.android.calendar/events

Share text ACTION_SEND putExtra(Intent.EXTRA_TEXT, text)

Always use resolveActivity() before startActivity() when the question expects safe handling.

15. EXAM REUSING RULE

Do not memorize every question as a completely different program. Memorize the master pattern and replace only the question-specific parts.

Master part Change according to question

EditText fields Name, email, mobile, patient name, product name, username, etc.

RadioButtons Gender, payment method, room type, category, etc.

CheckBoxes Hobbies, facilities, features, skills, add-ons, etc.

Spinner array Course, state, country, category, vehicle, room, etc.

ListView array Products, courses, tasks, contacts, bookings, books, etc.

Messages Validation errors, Toast messages, dialog titles and confirmations.

```
Intent destination Website, dialer, SMS, email, map, calendar or another Activity.
SharedPreferences keys Use meaningful keys matching the question.
```

16. FINAL MEMORIZATION CHAIN

Activity 1

↓

findViewById()

↓

Read values with getText().toString().trim()

↓

Validate with isEmpty(), length, setError()

↓

RadioButton / CheckBox / Spinner / AutoComplete / RatingBar

↓

TextWatcher / IME Done / DatePicker / TimePicker

↓

```
SharedPreferences save/read
```

↓

AlertDialog / Custom Dialog

↓

```
Bundle + Explicit Intent
```

↓

Activity 2

↓

Read getIntent().getExtras()

↓

Read SharedPreferences

↓

ListView + ArrayAdapter

↓

Normal click → AlertDialog + Toast

↓

Long press → Context Menu

↓

Delete → notifyDataSetChanged()

↓

Implicit Intent

├── Browser

├── Dialer

├── SMS

├── Email

├── Maps

├── Calendar

└── Share

↓

Fragments

↓

FragmentManager → replace() → addToBackStack()

↓

setArguments() → getArguments()

↓

Dynamic UI → addView()

17. IMPORTANT CORRECTION BEFORE EXAM USE

The master pattern is intentionally comprehensive, but when writing an individual lab answer, do not dump every feature into the program unless the question asks for it. Use the smallest correct subset that satisfies the question, while reusing the same syntax patterns.

Also remember: if your project uses AndroidX fragments, keep the entire Fragment implementation consistently on AndroidX. If your lab environment uses the platform Fragment API, keep it consistently on android.app.Fragment. Mixing the two APIs causes compile errors.

Exam strategy: First write the XML IDs, then Java findViewById(), then listeners, validation, data transfer and the required feature. Finally check imports and manifest entries.

MASTER VARIATION LIBRARY — ADD THESE WHEN THE QUESTION DEMANDS THEM

Password visibility:

```
showPasswordCheckBox.setOnCheckedChangeListener(
    (buttonView, checked) -> {
        if (checked) {
            passwordEditText.setInputType(
                android.text.InputType.TYPE_CLASS_TEXT);
        } else {
            passwordEditText.setInputType(
                android.text.InputType.TYPE_CLASS_TEXT |
                android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD);
        }
        passwordEditText.setSelection(
            passwordEditText.length());
    });
```

BMI:

```
double weight = Double.parseDouble(
    weightEditText.getText().toString());
double heightCm = Double.parseDouble(
    heightEditText.getText().toString());

double heightM = heightCm / 100.0;
double bmi = weight / (heightM * heightM);

String category;
if (bmi < 18.5) category = "Underweight";
else if (bmi < 25) category = "Normal";
else if (bmi < 30) category = "Overweight";
else category = "Obese";
```

Feedback Submit condition:

```
boolean enoughComment =
    commentEditText.getText().toString().trim().length() >= 10;

boolean ratingGiven =
    ratingBar.getRating() > 0;

submitButton.setEnabled(
    enoughComment && ratingGiven);
```

Remember Me session:

```
if (sp.getBoolean("isLoggedIn", false)) {
    startActivity(
        new Intent(this, HomeActivity.class));
    finish();
    return;
}

// successful login
sp.edit()
  .putBoolean("isLoggedIn", true)
  .putString("user", username)
  .apply();
```

Logout:

```
getSharedPreferences(
    "Session", MODE_PRIVATE)
    .edit()
    .clear()
    .apply();

Intent i = new Intent(
    this, LoginActivity.class);

i.setFlags(
    Intent.FLAG_ACTIVITY_NEW_TASK |
    Intent.FLAG_ACTIVITY_CLEAR_TASK);

startActivity(i);
```

Contact utility:

```
// Dial
openExternal(new Intent(
    Intent.ACTION_DIAL,
    Uri.parse("tel:" + phone)));

// SMS
openExternal(new Intent(
    Intent.ACTION_SENDTO,
    Uri.parse("sms:" + phone)));

// Email
Intent mail = new Intent(
    Intent.ACTION_SENDTO);
mail.setData(Uri.parse("mailto:" + email));
mail.putExtra(
    Intent.EXTRA_SUBJECT, "Subject");
mail.putExtra(
    Intent.EXTRA_TEXT, "Body");
openExternal(mail);

// Website
openExternal(new Intent(
    Intent.ACTION_VIEW,
```

```
    Uri.parse("https://example.com")));

// Share
Intent share = new Intent(Intent.ACTION_SEND);
share.setType("text/plain");
share.putExtra(Intent.EXTRA_TEXT, "Details");
startActivity(Intent.createChooser(
    share, "Share using"));
```

Cascading Spinner:

```
countrySpinner.setOnItemSelectedListener(
    new AdapterView.OnItemSelectedListener() {
        @Override
        public void onItemSelected(
                AdapterView<?> parent,
                View view,
                int position,
                long id) {

            String[] states =
                statesForCountry(position);

            stateSpinner.setAdapter(
                new ArrayAdapter<>(
                    MainActivity.this,
                    android.R.layout.simple_spinner_item,
                    states));
        }

        @Override
        public void onNothingSelected(
                AdapterView<?> parent) {}
    });
```

Fragment interface callback:

```
public interface OnColorSelected {
    void onColorSelected(int color);
}

@Override
public void onAttach(Context context) {
    super.onAttach(context);

    if (context instanceof OnColorSelected) {
        listener = (OnColorSelected) context;
    } else {
        throw new RuntimeException(
            "Activity must implement callback");
    }
}
```

PART 5 — ALL 59 QUESTIONS: WHAT TO CHANGE IN MASTER

Q1. Login Page

Question requirements: Username/password, validation, Toast, LinearLayout.

Master conversion: MainActivity form + validate + Toast.

Q2. Simple Calculator

Question requirements: Numbers, + − × ÷, result, clear.

Master conversion: Calculator block.

Q3. Explicit Intent

Question requirements: Activity 1 login → Activity 2.

Master conversion: openDashboard() + Bundle.

Q4. Activity Lifecycle

Question requirements: Seven callbacks + Logcat.

Master conversion: Lifecycle block.

Q5. RadioButton + CheckBox

Question requirements: One radio choice, multiple checkbox choices.

Master conversion: genderRadioGroup + checkboxes.

Q6. Implicit Intent

Question requirements: ACTION_VIEW website.

Master conversion: Browser method.

Q7. ListView

Question requirements: Array + ArrayAdapter + click Toast.

Master conversion: Dashboard ListView.

Q8. ScrollView

Question requirements: Dynamic TextViews + addView.

Master conversion: Main XML + dynamicContainer.

Q9. Spinner

Question requirements: Adapter + onItemSelected + onNothingSelected.

Master conversion: courseSpinner.

Q10. SharedPreferences

Question requirements: Store/read value.

Master conversion: preferences.

Q11. Student Registration Form

Question requirements: Name, email, mobile, gender, hobbies, display details.

Master conversion: Add fields + validation; keep master flow.

Q12. Secure Sign-Up

Question requirements: Username, strong password, confirm password, Show Password.

Master conversion: Password variation + validation.

Q13. BMI Calculator

Question requirements: Weight/height, numeric/range, category, NumberFormatException.

Master conversion: BMI variation.

Q14. Feedback + Counter

Question requirements: RatingBar, max150, TextWatcher, ≥10 chars, enable Submit.

Master conversion: Rating + TextWatcher.

Q15. ConstraintLayout Login

Question requirements: Logo, inputs, Login, Forgot password.

Master conversion: Replace main XML with ConstraintLayout while keeping Java patterns.

Q16. 4×4 Calculator

Question requirements: Digit/operator grid, =, Clear.

Master conversion: GridLayout + calculator method.

Q17. Profile Card

Question requirements: Banner, overlay, name/designation, Follow/Message.

Master conversion: Frame/Relative/Constraint layout variation.

Q18. Cascading Spinners

Question requirements: Country → State → City.

Master conversion: Dependent adapter variation.

Q19. To-Do List

Question requirements: Add task, click Toast, long delete.

Master conversion: ListView + notifyDataSetChanged.

Q20. Exit + Quick Settings

Question requirements: Back confirmation; Light/Dark/Blue single-choice.

Master conversion: onBackPressed/back handling + AlertDialog.

Q21. Contact Popup Menu

Question requirements: Call/Edit/Delete per row.

Master conversion: PopupMenu + AlertDialog delete.

Q22. Custom Dialog Item

Question requirements: Floating +, name/quantity, custom Toast icon.

Master conversion: Custom dialog + custom toast.

Q23. Tab Fragments

Question requirements: Home/Courses/Contact; replace; back stack.

Master conversion: Fragment buttons + loadFragment().

Q24. Activity → Fragment

Question requirements: Name/age Bundle; eligibility.

Master conversion: DetailsFragment setArguments/getArguments.

Q25. Two-Fragment Colour Picker

Question requirements: A buttons → B color via Activity interface.

Master conversion: Interface callback variation.

Q26. Appointment

Question requirements: Name/date/time, block past, IME Done, confirmation.

Master conversion: Date/time + IME.

Q27. Remember Me

Question requirements: Auto-fill username, Home, logout, skip login.

Master conversion: SharedPreferences session.

Q28. Contact Utility

Question requirements: Dial/SMS/email/website/share.

Master conversion: Implicit intent library.

Q29. Integrated contact/session variation

Question requirements: Combine the reusable contact/session patterns.

Master conversion: Master architecture.

Q30. Patient Appointment & Clinic Dashboard

Question requirements: Name/age/token-date; save name; doctor ListView; click dialog/Toast; long Context Menu → browser/map.

Master conversion: Rename fields + maps/browser.

Q31. Smart Fitness Tracker & Workout Portal

Question requirements: Username/target weight/calorie; workout ListView; long browser.

Master conversion: Rename form/list.

Q32. Student Course Enrollment & Academic Hub

Question requirements: Admission/department/contact; elective ListView; long browser syllabus.

Master conversion: Rename form/list.

Q33. Hotel Room Booking & Concierge Desk

Question requirements: Guest/ID/duration; amenities ListView; long maps.

Master conversion: Rename form/list.

Q34. Corporate Job Application & Talent Portal

Question requirements: Candidate fields; job openings; external browser.

Master conversion: Rename form/list.

Q35. Vehicle Rental System & Drive Dashboard

Question requirements: Rental fields; vehicle list; external map/browser.

Master conversion: Rename form/list.

Q36. Culinary Recipe Finder & Chef Kitchen

Question requirements: Recipe/category fields; recipe list; browser.

Master conversion: Rename form/list.

Q37. E-Commerce Product Launch & Pre-Order

Question requirements: Product/customer/order fields; product list; external action.

Master conversion: Rename form/list.

Q38. Travel Itinerary Planner & Tourism Guide

Question requirements: Traveller/destination/date; destination list; Maps.

Master conversion: Date + Maps variation.

Q39. Alumni Network & Placement Connect

Question requirements: Alumni/profile/contact; opportunities list; external action.

Master conversion: Rename form/list.

Q40. Custom Typography Studio

Question requirements: Typography configuration UI + persistence + list/menu patterns.

Master conversion: Use master layout and replace widgets.

Q41. Inventory Reorder Deck

Question requirements: Inventory/reorder fields + list + menus.

Master conversion: List + validation + notify.

Q42. Real Estate Agent Connector

Question requirements: Property/agent data + maps/browser.

Master conversion: External intent variation.

Q43. Secure Corporate Vault Browser

Question requirements: Vault/file/category + list + Context Menu.

Master conversion: List/context/browser.

Q44. Smart Travel Hub Link Handler

Question requirements: Destination/link data + external navigation.

Master conversion: ACTION_VIEW/Maps.

Q45. Freelance Project Bidding Board

Question requirements: Project/bid form + project list.

Master conversion: Validation/preferences/list.

Q46. Gym Trainer Allocation Portal

Question requirements: Trainer/fitness controls, radio/check boxes, list.

Master conversion: Widgets + master flow.

Q47. Vehicle Service Scheduler

Question requirements: Vehicle/service/date + list.

Master conversion: DatePicker + list.

Q48. E-Banking Payee Hub

Question requirements: Payee form + confirmation + list.

Master conversion: Dialog + validation + list.

Q49. Digital Library & Book Checkout

Question requirements: Book/member + list + checkout dialog.

Master conversion: Adapter + dialog.

Q50. Multi-Tier Fitness Assessment Console

Question requirements: Advanced UI mix: RadioButton, CheckBox, ProgressBar, Spinner/AutoComplete.

Master conversion: Advanced widget variation.

Q51. Smart Vehicle Fleet Broker

Question requirements: Fleet/vehicle structured UI + ListView + external map/browser.

Master conversion: Master flow.

Q52. Professional Flight Booking & Cargo Gateway

Question requirements: Passenger/flight/cargo structured form + list/actions.

Master conversion: Master flow + validation.

Q53. E-Learning Registration & Progress Monitor

Question requirements: Registration + ProgressBar + course list.

Master conversion: Progress + list.

Q54. Tele-Health Diagnostic Portal

Question requirements: Patient/diagnostic form + advanced widgets + dashboard.

Master conversion: Validation + preferences + list.

Q55. Retail Point-of-Sale Checkout Hub

Question requirements: GridLayout invoice, product Spinner, quantity boundaries, RadioButtons, menu transaction, invoice ListView, external map/shipping.

Master conversion: Grid + Spinner + numeric validation + list.

Q56. Corporate Recruitment Portal & Skills Matrix

Question requirements: Skills CheckBoxes, language RadioGroup, eligibility AlertDialog, job list, browser.

Master conversion: CheckBox + RadioGroup + dialog.

Q57. Culinary Order & Home Delivery Logistics

Question requirements: AutoComplete location, food Spinner, validation, progress timeline, custom Adapter, maps.

Master conversion: AutoComplete + Spinner + ProgressBar + Adapter.

Q58. Academic Project Repository & Archive

Question requirements: Project metadata, advanced UI, persistence, list, menus, external document/browser.

Master conversion: Master architecture.

Q59. Smart Hotel Room Management Deck

Question requirements: Room/guest/booking controls, advanced UI, persistence, ListView, menus, external map/browser.

Master conversion: Master architecture.

The 59-question bank is intentionally a pattern bank: Q30–Q59 mostly change the domain vocabulary while reusing the same Activity 1 → validation → preferences → Bundle → Explicit Intent → Activity 2 → ListView → click → long press → Context Menu → Implicit Intent architecture.

PART 6 — FINAL VIVA

What is an Activity?

Answer: A screen managed by Android with a lifecycle.

Why setContentView()?

Answer: It loads the XML layout so its Views can be found and displayed.

Why super.onCreate()?

Answer: It lets the parent Activity perform its normal initialization.

What is onCreate?

Answer: Initial creation and initialization.

What is onStart?

Answer: The Activity becomes visible.

What is onResume?

Answer: The Activity is in the foreground and ready for interaction.

What is onPause?

Answer: The Activity is losing foreground focus.

What is onStop?

Answer: The Activity is no longer visible.

What is onRestart?

Answer: A stopped Activity is returning.

What is onDestroy?

Answer: The Activity is being destroyed; do not use it as the only critical persistence point.

What is Logcat?

Answer: Android's logging/debug output area.

RadioButton vs CheckBox?

Answer: One choice in a RadioGroup versus independent multiple choices.

What is an Adapter?

Answer: A bridge between data and a UI list/drop-down.

Why notifyDataSetChanged()?

Answer: It tells the Adapter its underlying data changed so the UI can refresh.

What is SharedPreferences?

Answer: Small persistent key-value storage.

What is Bundle?

Answer: A key-value container for small data between components.

Explicit Intent?

Answer: The exact target component is named.

Implicit Intent?

Answer: Action/data is specified and Android resolves a suitable handler.

Why resolveActivity()?

Answer: To avoid starting an Intent when no handler exists.

What is a Fragment?

Answer: A reusable UI portion hosted inside an Activity.

What does replace() do?

Answer: Replaces the Fragment in a container.

What does addToBackStack() do?

Answer: Records a Fragment transaction so Back can return to the previous state.

setArguments()/getArguments()?

Answer: Send/retrieve a Fragment Bundle.

What is TextWatcher?

Answer: A listener mechanism for observing editable text changes.

What is IME?

Answer: Input Method Editor / on-screen keyboard.

How block past dates?

Answer: Use DatePickerDialog.getDatePicker().setMinDate(System.currentTimeMillis()).

What is Toast?

Answer: A temporary message that disappears automatically.

What is AlertDialog?

Answer: A modal message, confirmation or choice dialog.

What is Context Menu?

Answer: Actions for a selected View, commonly opened by long press.

What is PopupMenu?

Answer: A small menu anchored to a View.

What is Options Menu?

Answer: Activity-level actions/settings.

Why one child in ScrollView?

Answer: Use one direct container child, normally a LinearLayout, for all scrolling content.

Why try/catch in a calculator?

Answer: Invalid numeric strings can cause NumberFormatException.

How implement Remember Me?

Answer: Store a username/session flag in SharedPreferences and read it when the app starts.

Why finish() after login?

Answer: It prevents Back from returning to the login screen.

What is minSdk?

Answer: The minimum Android API level supported by the app.

What is a custom Adapter?

Answer: An Adapter that binds custom objects to custom row XML.

What is a cascading Spinner?

Answer: One Spinner's selection changes the data/adapter of the next Spinner.

What is resolveActivity()?

Answer: A check for whether Android has an installed handler for the Intent.

Why use notifyDataSetChanged() after delete?

Answer: Because the underlying ArrayList changed and the Adapter must refresh its displayed rows.

PART 7 — COMMON ERRORS + FIXES

R.id cannot be resolved

Cause: ID is missing/misspelled or XML has another error. Fix: Compare XML and Java IDs exactly; fix red XML errors; rebuild.

NullPointerException from findViewById

Cause: Wrong layout/ID or findViewById was called before setContentView(). Fix: Call setContentView() first and verify the ID exists in that layout.

Activity not found

Cause: Activity is not declared or class/package name is wrong. Fix: Declare the Activity in AndroidManifest.xml and verify the exact class name.

Implicit Intent crash

Cause: No installed app can handle the Intent. Fix: Call resolveActivity() before startActivity(); otherwise show Toast.

RadioGroup returns -1

Cause: No RadioButton is selected. Fix: Check id == -1 before findViewById(id).

ListView does not refresh

Cause: The ArrayList changed but Adapter was not notified. Fix: Call adapter.notifyDataSetChanged().

Spinner is empty

Cause: Adapter was not attached or data is empty. Fix: Create adapter and call spinner.setAdapter(adapter).

ScrollView does not scroll

Cause: Wrong hierarchy or multiple direct children. Fix: Use one direct child, normally a vertical LinearLayout.

Dynamic View does not appear

Cause: View was created but not added to a visible parent. Fix: Call parent.addView(view).

Fragment View is null

Cause: Wrong root or View accessed before inflation. Fix: Inflate the Fragment layout and use the returned root/onViewCreated().

Fragment transaction fails

Cause: Wrong container or mixed platform/AndroidX Fragment APIs. Fix: Use androidx.fragment.app.Fragment consistently and call commit().

SharedPreferences returns default

Cause: Key mismatch or value never saved. Fix: Compare put/get keys exactly and call apply().

TextWatcher behaves unexpectedly

Cause: Its callback runs while the user types. Fix: Keep live logic small and use the correct callback.

Past dates are selectable

Cause: Minimum date was not set. Fix: Use getDatePicker().setMinDate(System.currentTimeMillis()).

IME Done does not trigger

Cause: XML imeOptions and Java action ID do not match. Fix: Use actionDone with EditorInfo.IME_ACTION_DONE.

Menu ID error

Cause: Java references a different menu item ID. Fix: Compare res/menu IDs and R.id references exactly.

Custom dialog field is null

Cause: findViewById was called on the Activity instead of the inflated dialog View. Fix: Use view.findViewById().

Calculator crashes

Cause: Invalid text is passed to parseDouble/parseInt. Fix: Trim input and parse inside try/catch.

Division by zero

Cause: Divisor is zero. Fix: Check divisor before division.

Long press operates on wrong row

Cause: Selected position was not retained. Fix: Store selectedPosition during onItemLongClick.

Back returns to Login

Cause: Login Activity remains in the back stack. Fix: Call finish() after successful login; use task-clearing flags for logout when required.

Lambda syntax does not compile

Cause: Project/language configuration does not support lambda or the target is not a functional interface. Fix: Use the anonymous-class form shown in the side-by-side section.

BRUTE-FORCE PRACTICAL RECOVERY

```
1. BUILD / COMPILE FIRST.
2. Confirm setContentView().
3. Confirm every R.id exists.
4. Add ONE button → Toast.
5. Add validation.
6. Add storage.
7. Add Intent.
8. Add ListView + Adapter.
9. Add normal click.
10. Add long-click + Context Menu.
11. Add external Intent + resolveActivity().
12. Add Fragment only if required.
13. Build/run again.
14. Demonstrate.
```

Never add all features at once while debugging. Build a working skeleton first, then add one required variation at a time.

FINAL 30-SECOND CHECKLIST

```
[ ] Java + XML
[ ] Min SDK acceptable
[ ] setContentView()
[ ] Every R.id matches XML
[ ] Validation
[ ] Toast
[ ] AlertDialog
[ ] RadioButton / CheckBox if required
[ ] Spinner / Adapter if required
[ ] SharedPreferences
[ ] Bundle / Explicit Intent
[ ] Activity declared in Manifest
[ ] ListView + Adapter
[ ] notifyDataSetChanged()
[ ] Long-click / Context Menu
[ ] resolveActivity() for external Intent
[ ] FragmentManager / replace / back stack if required
[ ] DatePicker / TimePicker if required
[ ] IME Done if required
[ ] No SQLite
[ ] Test before calling examiner
```

FINAL MEMORY SENTENCE

INPUT → VALIDATE → STORE → CONFIRM → SEND → DISPLAY → CLICK → LONG CLICK → ACTION.

The first theory book explicitly frames the exam around reusable building blocks, and the simple/master artifacts explicitly say to change field names, arrays, messages and destinations rather than memorizing separate architectures. filecite turn18file0 L11-L14 filecite turn18file1 L551-L554